On the Closure of Compositional Hierarchies in Digital Symmetries

A rigorous researcher-engineer bridging abstract mathematics and concrete digital systems.

June 12, 2026

Contents

1 Title Page(s)

On the Closure of Compositional Hierarchies in Digital Symmetries

A technical book on compositionality, closure properties, and symmetries in digital systems

Technical Draft

Version 0.1.0

Status: draft

Created: 2026-06-12

Updated: 2026-06-12

Author: *TBD*

Prepared for reproducible, proof-oriented development

On the Closure of Compositional Hierarchies in Digital Symmetries

Technical Draft

A technical book on compositionality, closure properties, and symmetries in digital systems

Version 0.1.0

Status: draft

Created: 2026-06-12

Updated: 2026-06-12

Author: *TBD*

Draft prepared for reproducible, proof-oriented development

2 Front Matter

2.1 Preface

This book is motivated by a simple but far-reaching observation: many digital systems are built compositionally, yet their behavior is often understood only locally. When composition is organized well, it can exhibit closure properties that make whole families of constructions stable under reuse, refinement, and abstraction. At the same time, symmetry is not merely a decorative feature of such systems; it can reveal invariants, reduce redundancy, and expose canonical forms that are otherwise hidden. The central aim of this book is to develop a unified language for these ideas and to show how they interact in concrete digital settings.

The intended audience includes readers with a working background in discrete mathematics, basic category theory, and digital logic. The exposition assumes familiarity with elementary proof techniques, Boolean reasoning, and the idea of compositional design. It is also written for practitioners who want a formal framework that connects abstract structure to buildable artifacts in hardware, software, and related computational systems. Where possible, the book keeps the mathematics explicit and the constructions executable.

The book is organized around a "proofs-as-builds" workflow. Claims are treated as objects to be constructed, checked, and reused rather than as isolated statements to be read once and set aside. In that spirit, code serves as witness: implementations, derivations, and formal artifacts are intended to certify the mathematical content of the surrounding text. Each chapter is designed to stand with its own artifacts, examples, and verification material, so that the reader can inspect not only the result but also the path by which it is obtained.

This usage model is deliberate. Readers may proceed linearly from foundations to applications, or they may consult individual chapters as self-contained modules when working on a specific topic such as closure operators, modular design, symmetry analysis, or algorithmic equivalence checking. The front matter supplies notation and methodological conventions; the main chapters develop the theory and its applications; and the back matter collects supporting materials for reproducibility, reference, and revision.

The overarching goal is not merely to catalog results, but to provide a framework in which compositional hierarchies, closure phenomena, and symmetry principles can be studied together. If successful, the book will help the reader move between abstract structure and concrete implementation with minimal friction, and will provide a set of reusable artifacts for further work in formal methods, digital design, and symmetry-aware computation.

2.2 Notation & Conventions

This book uses a small set of symbols and conventions consistently across chapters, with chapter-local redefinitions when a later argument benefits from a narrower or more specialized meaning. When a symbol is first introduced in a chapter, it should be understood in that chapter's local context unless explicitly stated otherwise.

For Boolean objects, we write

$$\mathbb{B} \coloneqq \{0, 1\}$$

for the Boolean domain, and

$$\mathbb{B}^n$$

for the set of bitstrings of length n. The notation

$$\mathrm{Bool}(n)$$

denotes the set of all n-ary Boolean functions, that is, functions $f : \mathbb{B}^n \to \mathbb{B}$.

We use the following structural symbols throughout the book. Let $\mathcal{O}$ denote an operad, and let $\mathcal{O}(n)$ denote the operations of arity n. The symmetric group on n letters is written Σ_n. Composition of operations, or serial substitution, is written $\circ$. Parallel or monoidal composition is written $\otimes$. Feedback or trace is written Tr. Closure operators are written $\mathrm{cl}(\cdot)$. When discussing algebraic closure systems, a *clone* means a set of operations closed under projections and composition. Group actions are written $G \curvearrowright X$, and automorphism groups are written $\mathrm{Aut}(\cdot)$.

Typing conventions are explicit: ports are typed, and diagrams may be colored to indicate types or object labels. In particular, when a chapter uses colored operads or typed diagrams, the color set should be read as part of the typing discipline rather than as decorative notation. This convention is intended to keep interface data visible in every compositional construction, especially when a later chapter specializes the ambient category, operad, or diagrammatic language.

Error and resolution metrics are chapter-specific but follow a common default convention. We write $\varepsilon(r)$ for a resolution- or error-dependent quantity, with the understanding that the precise meaning of r and the metric itself may be overridden in a given chapter. Unless otherwise stated, the default error metric is normalized Hamming distance. For example, for $f, g \in \mathrm{Bool}(n)$, we define

$$\varepsilon_{\mathrm{tt}}(r = n; f, g) = \frac{1}{2^n} \sum_{x \in \mathbb{B}^n} \mathbf{1}[f(x) \neq g(x)],$$

where $\mathbf{1}[\cdot]$ is the indicator function. This convention measures the fraction of inputs on which two Boolean functions disagree.

Individual chapters may override the default metric when the application requires a different notion of discrepancy, such as bisimulation distance, metastability-aware error, or another domain-specific resolution criterion. In such cases, the chapter should state the override explicitly and use it consistently within that chapter.

Unless otherwise noted, all notation is intended to be read in a proof-oriented, composition-first manner: symbols denote objects, operations, or metrics that are stable under the closure and symmetry constructions developed in later chapters.

2.3 Methodological Protocol

Claims-as-builds. This book adopts a claims-as-builds workflow: each substantive claim is paired with an executable artifact, test, or check that can be run, inspected, or reproduced. In practice, a theorem is not treated as complete until it is accompanied by the corresponding proof object, derivation script, verification harness, or other machine-checkable witness appropriate to the topic at hand. This convention keeps the exposition tightly coupled to reproducible evidence and makes the logical dependencies between results explicit.

Acquire–Calibrate–Log. The working protocol for examples, experiments, and derived results has three stages. First, acquire the relevant specifications, datasets, or source artifacts. Second, calibrate the chapter-specific error model $\varepsilon(r)$ at the appropriate resolution r, so that approximations, tolerances, and comparison criteria are stated consistently across the book. Third, log reproducibility metadata, including hashes, random seeds, toolchains, and other build parameters needed to reconstruct the result. This protocol is used uniformly for analytic derivations, computational experiments, and build logs.

Reading map. The book is organized as a dependency graph rather than a strictly linear narrative, so readers may enter from different backgrounds and follow different paths through the

material. The reading map indicates which chapters depend on which earlier constructions, allowing a reader to move from foundations to applications, or to jump directly to a topic of interest while consulting prerequisite material as needed. The map is meant to support both sequential reading and selective reference use, with the dependency structure serving as a guide to conceptual prerequisites and technical reuse.

Usage convention. Throughout the book, when a result is stated, the accompanying artifact should be understood as part of the claim. When a result is approximate, the relevant calibration and error bookkeeping should be stated explicitly. When a result is reproduced, the logged metadata should be sufficient to recover the same computational environment and verify the same outputs. Readers should therefore expect proofs, code, data, and build records to function together as the evidential basis for the text.

How to read this book. For readers with a mathematical background, the recommended path is to begin with the foundational chapters, then follow the dependency graph into modularity, feedback, and symmetry. For readers coming from engineering or implementation practice, it is often useful to start with the usage conventions here, consult the notation chapter for symbols and error metrics, and then move into the chapters whose artifacts are most directly relevant to the problem at hand. In either case, the reading map is intended to make prerequisite structure explicit without forcing a single linear route through the material.

3 Chapters

3.1 Part I: Foundations

3.1.1 Orientation & Thesis: Closure of Compositional Hierarchies in Digital Symmetries

This chapter orients the reader to the book's central thesis: that many digital systems admit a useful notion of *closure* under composition, and that the resulting compositional hierarchies are often organized by symmetry. The guiding question is not merely whether a given object can be built from simpler parts, but whether the class of admissible objects is stable under the operations used to build them, and whether the resulting structure can be analyzed up to the symmetries that preserve its behavior.

We will use the term *digital object* broadly to include Boolean functions on $\mathbb{B}^n$, circuits, finite automata, and codes. These objects appear in different guises across logic, hardware, software, and information theory, but they share a common feature: they are finite or finitely presented systems whose behavior can be described, transformed, and compared by explicit operations. In this setting, closure means that a chosen class of objects is preserved by the relevant constructions, while hierarchy refers to the stratification of objects by compositional depth, structural complexity, or refinement level.

A recurring theme of the book is that symmetry is not an afterthought but a structural constraint. We will study invariance and equivariance under permutations Σ_n, under input and output negations, and under the combined action that gives rise to NPN classes. These symmetry notions organize Boolean functions and circuits into equivalence classes that are often more informative than raw syntactic descriptions. In later chapters, the same perspective will be extended to automata, dataflow systems, and other compositional formalisms.

Composition is the second organizing principle. The book will repeatedly use three basic operations: sequential composition $\circ$, parallel composition $\otimes$, and feedback or trace Tr. The intended laws are familiar but essential: associativity for sequential and parallel composition, symmetry for interchange of components, and functoriality or naturality conditions that ensure compatibility between structure-preserving maps and the operations themselves. These laws provide the algebraic backbone for the constructions developed throughout the book.

To compare digital objects at different levels of description, we will also use error and discrepancy measures. At the level of Boolean functions, $\varepsilon(r)$ may denote truth-table Hamming error at resolution r; at the level of circuits, it may denote a structural edit distance for wiring at resolution r. The precise form of ε will vary with context, but the underlying idea is consistent: a hierarchy is meaningful only if we can quantify how far an object lies from another object, or from a target specification, at the chosen level of abstraction.

The scope of this chapter is intentionally foundational. It does not attempt to prove the deepest classification theorems or to develop every formalism in full generality. Instead, it establishes the vocabulary and the thesis that will govern the rest of the book: digital systems are best understood as compositional objects equipped with symmetries, and the central technical problem is to determine when the resulting classes are closed under the operations that generate them. Once this problem is posed clearly, the later chapters can specialize it to operads, PROPs, clones, modular design, feedback, and applications in computation and engineering.

In short, the book asks three questions throughout. What are the relevant digital objects? Which symmetries identify behaviorally equivalent forms? And under which composition laws do these objects form closed hierarchies? The chapters that follow develop these questions into precise definitions, constructions, and examples.

For orientation, the next chapter develops the categorical and algebraic foundations of composition itself, including operads, PROPs, and symmetric monoidal structure. That material will supply the formal language needed to state closure properties precisely and to compare different notions of hierarchy across the book's later case studies.

For the purposes of this book, *closure* will mean more than mere closure under a single operation. A class of digital objects is closed when the operations that generate, combine, or transform its members do not force us to leave the class in order to describe the result. Likewise, *hierarchy* will mean more than a simple ordering by size: it will refer to a layered organization in which objects at one level are assembled from objects at lower levels, and where the passage between levels is controlled by explicit composition laws and symmetry relations. These two ideas—closure and hierarchy—will recur throughout the book as the main criteria for deciding whether a formalism is robust enough to support analysis, synthesis, and comparison.

The chapters ahead will therefore proceed in a deliberately cumulative way. We begin with the abstract language of composition, then specialize to Boolean functions and their clone-theoretic closures, and then move outward to circuits, automata, modular design, and symmetry-aware applications. The unifying claim is that the same structural questions reappear in each setting: what is preserved, what is identified, and what is generated when digital systems are composed?

3.1.2 Operads, PROPs, and Symmetric Monoidal Foundations

Operads provide a compact language for describing how multi-input operations compose. In the simplest setting, an operad consists of operations with a specified arity, a unit operation, symmetric group actions that permute inputs, and a substitution law that allows one operation to be inserted into another. This makes operads well suited to modeling syntax with binding or branching structure, where the essential data are not merely the operations themselves but the admissible ways

they can be assembled.

A useful way to read an operad is as a theory of compositional patterns. The substitution operation expresses the passage from local pieces to larger expressions, while the unit records the identity pattern for composition. Symmetric actions encode the fact that the order of inputs may be irrelevant up to permutation, or relevant only up to a specified symmetry. An operad algebra then interprets these abstract operations in a concrete domain: each formal operation is sent to an actual operation on a set, space, or other carrier, and the operadic laws ensure that interpretation respects composition.

This perspective naturally separates syntax from semantics. Free operads generate formal composites from a chosen collection of generators, producing the most general syntax consistent with the operadic laws. Quotienting then imposes equations or identifications, collapsing syntactically distinct expressions that are semantically equivalent for the intended application. In this way, operads support a build-and-refine workflow: first generate the space of admissible composites, then impose relations to obtain the desired notion of equivalence.

PROPs extend this idea from single-output operations to operations with multiple inputs and multiple outputs. A PROP may be viewed as a strict symmetric monoidal category whose objects are natural numbers, with tensor product given by addition. Morphisms in a PROP can therefore represent circuits, wiring diagrams, or other process networks with several input and output ports. The monoidal structure captures parallel composition, while categorical composition captures sequential connection of outputs to inputs.

From this viewpoint, a circuit is not merely a graph but a morphism with typed boundary data. Wiring diagrams become the graphical calculus for composing such morphisms, and the PROP axioms guarantee that different parenthesizations and orderings of independent components agree up to the prescribed symmetry. This makes PROPs a natural foundation for digital systems, where one often needs to reason simultaneously about fan-in, fan-out, and the interaction of many wires.

The relation between operads and PROPs is complementary. Operads emphasize operations with many inputs and one output, which is often the right abstraction for syntax trees, term formation, and hierarchical composition. PROPs generalize to many-to-many processes, which is better suited to circuits, signal flow, and networked systems. Both structures encode compositionality, but at different levels of interface complexity.

Colored operads refine the operadic picture by allowing operations to carry typed ports. Instead of a single undifferentiated notion of input, each input and output is assigned a color, and composition is permitted only when the colors match appropriately. This typed discipline is especially useful for modular systems, where components may have heterogeneous interfaces and where correctness depends on respecting port types. Colored operads therefore provide a bridge between abstract composition and typed engineering practice.

Colored operads are also closely related to Lawvere theories. Both frameworks describe algebraic theories in terms of operations and equations, but they organize the data differently. Lawvere theories emphasize finite-product structure and algebraic operations on tuples, while colored operads emphasize typed multi-input operations and substitution. In many applications, these viewpoints can be translated into one another, and the choice between them depends on whether the primary concern is product-based algebraic structure or typed compositional syntax.

For the purposes of this book, the central role of operads, PROPs, and colored operads is foundational. They supply the language in which compositional systems can be generated, interpreted, and compared. They also prepare the ground for later chapters on closure, modularity, and symmetry by making explicit the algebraic forms of composition that underlie digital hierarchies.

A recurring theme is observational equivalence. For PROP morphisms, one may introduce a

relation $\varepsilon(r)$ to denote observational equivalence at a resolution or regime parameter r. In the default logical setting, this may be taken as ordinary logical equivalence at $r = \text{spec}$, where spec indicates the specification level. The point of such a notation is to distinguish raw syntactic identity from the equivalence induced by the chosen observational context. Two morphisms may differ as expressions while remaining indistinguishable under the relevant semantics.

This distinction between syntax and observation is not merely philosophical. It is the mechanism by which formal systems become usable in practice. One first constructs a free compositional object, then identifies the relations that matter at the chosen level of observation, and finally interprets the resulting quotient in a concrete semantic domain. Operads and PROPs thus serve as the algebraic infrastructure for the book's broader program: to understand how compositional hierarchies close under the symmetries and equivalences that arise in digital systems.

To summarize the chapter-level role of these notions: operads capture substitutional composition, PROPs capture multi-input/multi-output wiring, and colored operads capture typed interfaces. Together they provide a common foundation for syntax, semantics, and equivalence in compositional models. This foundation will be used repeatedly in later chapters whenever a system is assembled from generators, constrained by equations, or compared up to an observational notion of sameness.

3.1.3 Closure & Clones: Post's Lattice for \(\mathrm{Bool}\)

Closure operators provide the language for describing when a family of Boolean functions is stable under composition. In the Boolean setting, the ambient set is $\text{Bool} = \{0, 1\}$, and the central objects are sets of operations $F \subseteq \bigcup_{n\geq 1} \text{Bool}^{\text{Bool}^n}$ that are closed under superposition and contain the projections. The projections are the basic coordinate maps $\pi_i^n(x_1, \ldots, x_n) = x_i$, and they serve as the identity-like generators for composition. A family with these properties is called a *clone*.

A closure operator on a collection of Boolean operations is typically understood as a map Cl satisfying extensivity, monotonicity, and idempotence. In this chapter, closure is used in the algebraic sense: given a set of Boolean functions, one forms the smallest clone containing it by repeatedly adjoining all functions obtainable by composition with projections and previously obtained operations. This closure process organizes Boolean function theory into a lattice of classes ordered by inclusion. The lattice order is inclusion of clones, and the closure of a set is the least clone containing it.

Post's lattice is the classification of all clones of Boolean functions. It is a finite, highly structured lattice whose nodes are the closed classes of Boolean operations on Bool. Among the most important named clones are the monotone clone M, the affine clone L, the self-dual clone D, and the classes preserving constants or distinguished values, often denoted by T_0 and T_1. These classes capture familiar semantic constraints: monotonicity, linearity over $\mathbb{F}_2$, self-duality, and preservation of 0 or 1. In practice, they provide the first line of attack for deciding whether a given basis can generate all Boolean functions.

The lattice viewpoint makes the notion of *precomplete* classes precise. A clone is precomplete if it is maximal among proper clones, meaning that adjoining any operation not already in the clone yields the full clone of all Boolean functions. Post's theorem identifies the precomplete classes and thereby reduces many questions about Boolean expressibility to membership in one of a small number of maximal closed families. In practice, this means that to show a set of functions is not functionally complete, it suffices to show that it lies inside one of these maximal proper clones. Equivalently, to prove completeness, it is enough to show that the set escapes every precomplete class.

Functional completeness is the property that a set of Boolean functions generates all Boolean

functions under composition. In the language of clones, a set is functionally complete exactly when its generated clone is the full clone $\mathrm{Bool}^{\mathrm{Bool}^{<\omega}}$. Classical criteria due to Sheffer show that a single binary connective such as NAND or NOR is functionally complete. More generally, a set of operations is complete if it is not contained in any proper precomplete clone. This gives a decision procedure in principle: test whether the candidate set lies inside one of the known maximal closed classes. The Sheffer criteria are the canonical examples of a basis that avoids all of the standard obstructions at once.

The imported error metric $\varepsilon(r)$ can be read as a comparison score for Boolean representations, combining truth-table Hamming distance with a penalty for variable mismatch. Such a metric is useful when comparing candidate implementations, normal forms, or generated circuits against a target specification. In the present chapter, it plays the role of a practical evaluation tool alongside the algebraic closure theory: one measures how far a proposed function or realization is from a desired Boolean behavior, while the clone-theoretic framework determines whether exact realization is possible by composition. In that sense, $\varepsilon(r)$ is a quantitative companion to the qualitative clone classification.

Algorithmically, clone theory supports several standard tasks. Membership tests ask whether a given Boolean function belongs to a specified clone such as M, L, D, T_0, or T_1. Minimal generating set problems ask for the smallest family of operations whose closure is a target clone or the full Boolean clone. Canonicalization and equivalence procedures often use NPN classes, where functions are identified up to negation of inputs, permutation of variables, and negation of the output. These reductions are especially useful in automated reasoning and circuit synthesis, where many functions differ only by symmetry. In a workflow setting, one typically canonicalizes a candidate, checks clone membership, and then uses the result to infer completeness or incompleteness.

A useful way to read Post's lattice is as a hierarchy of semantic invariants. Monotone functions preserve the product order on $\{0,1\}^n$; affine functions are those representable by XORs and constants; self-dual functions satisfy $f(\bar{x}) = \overline{f(x)}$; and the T_0 and T_1 classes preserve the distinguished constants 0 and 1, respectively. Each such property defines a closed class, and intersections of these classes produce further clones. The lattice records exactly how these constraints combine and where they stop being closed under composition. This makes the classification simultaneously semantic and operational: each node corresponds to a recognizable behavior and to a closure condition.

From the standpoint of synthesis, the chapter's main operational question is whether a target Boolean behavior can be built from a chosen basis. If the basis lies inside a proper clone, then the answer is negative for every function outside that clone. If the basis escapes all proper precomplete clones, then it is complete and can generate every Boolean function. This dichotomy is the Boolean prototype for later chapters, where closure conditions will be studied in richer compositional settings. The Boolean case is especially valuable because the full classification is known and can be used as a reference model for more general closure hierarchies.

The chapter's organizing principle is that closure is not merely a property of a set of functions, but a design discipline. Once the relevant clone is identified, one can determine which operations are admissible, which are forbidden, and whether a target specification is expressible from a chosen basis. Post's lattice provides the global map; closure operators and clone membership provide the local mechanics; and functional completeness supplies the synthesis criterion. In this sense, the chapter links algebraic classification to practical circuit design and decision procedures.

In later chapters, these ideas will reappear in more structured settings, including modular composition, symmetry analysis, and algorithmic synthesis. Here they establish the Boolean baseline: the smallest closed families, the maximal proper ones, and the exact boundary between incomplete and complete functional bases. The result is a compact but complete toolkit for reasoning about Boolean expressibility, basis selection, and the algebra of composition.

3.2 Part II: Constructions and Modularity

3.2.1 Hierarchical Modularity & Contracts

This chapter develops the modular view of digital systems as collections of components with explicit interfaces, together with the contracts that govern their composition. The guiding idea is that a module is not merely a piece of implementation, but a reusable artifact whose behavior is constrained at its boundary. When modules are assembled hierarchically, the resulting system should preserve the invariants stated at each level of abstraction, and those invariants should be checkable by construction.

A useful starting point is the boundary error metric $\varepsilon(r)$, interpreted as the probability that a contract is violated at an interface or other composition boundary. Here r may be read as a resolution parameter, a refinement level, or a resource scale, depending on the surrounding model. The point of introducing $\varepsilon(r)$ is not to replace logical correctness with a probabilistic surrogate, but to make explicit how often a design departs from its declared specification when it is exercised at a given scale. In a hierarchical setting, one expects such boundary error to decrease under refinement, or at least to be controlled by a compositional bound.

Modules are characterized by their interfaces, parameterization, and reusability conditions. An interface specifies the data, signals, or morphisms that may cross the module boundary, while parameterization records the family of behaviors obtained by varying admissible inputs or configuration values. Reusability requires more than syntactic compatibility: a module should be substitutable in a larger assembly without invalidating the assumptions made by its environment. In contract-based design, this is expressed by a pair of obligations. The module assumes certain properties of its context and guarantees certain properties in return. Composition is valid when the guarantees of one component satisfy the assumptions of the next.

This perspective aligns naturally with categorical descriptions of open systems. In particular, hypergraph categories and decorated cospans provide a formal language for open circuits and other boundary-bearing structures. A decorated cospan represents a system together with designated input and output interfaces, while the decoration records the internal structure or semantics of the open component. Hypergraph categories supply the algebraic operations needed to compose such systems in parallel and in series, while retaining explicit boundary data. In this setting, contracts can be attached to interfaces and propagated through composition, so that the semantics of a larger circuit is assembled from the semantics of its parts.

The central technical requirement is that refinement maps preserve the intended invariants across levels of abstraction. A refinement map relates a coarse specification to a more detailed implementation, or one module version to a more concrete realization. If the map is well chosen, then properties proved at the abstract level remain valid after refinement, possibly after translation into the language of the lower level. This is the mechanism by which hierarchical invariants compose: one proves a property once at a high level, then shows that each refinement step respects the property, so that the final implementation inherits it.

In practice, hierarchical invariants are most useful when they are stable under the operations used to build systems. If two modules each satisfy their local contracts, then their composite should satisfy a derived contract. Likewise, if a module is refined into a more detailed implementation, the refined module should satisfy the original contract or a strengthening of it. These closure properties are the modular analogue of algebraic closure: the class of admissible components remains closed under the composition rules of the design language.

The chapter therefore treats contracts not as informal documentation, but as executable specifications that can be checked in a build pipeline. A contract-checked library is an artifact in which

module interfaces, refinement relations, and invariant checks are encoded so that continuous integration can validate them automatically. Such a library serves two purposes. First, it demonstrates that the abstract theory can be instantiated in a concrete engineering workflow. Second, it provides a reproducible testbed for studying how boundary errors, refinement maps, and compositional invariants behave under change.

The intended artifact for this chapter is a contract-checked library with CI tests. The library should expose a small collection of modules with explicit interfaces, a refinement relation between at least two abstraction levels, and automated tests that verify the stated contracts at composition boundaries. The tests should report whether the observed boundary behavior is consistent with the chosen $\varepsilon(r)$ criterion, and they should fail when a contract is violated. In this way, the chapter connects the categorical language of open systems with the practical discipline of buildable, checkable modular software and hardware design.

Taken together, these ideas establish the chapter's thesis: hierarchical modularity is strongest when interfaces are explicit, refinement is structure-preserving, and contracts are enforced by executable checks. Under those conditions, compositional invariants are not merely asserted; they are maintained across levels of abstraction and across the assembly of open components.

To make the discussion operational, it is useful to distinguish three layers of specification. At the outer layer, an interface declares the observable ports of a module and the admissible directions of information flow. At the middle layer, a contract states the assumptions on the environment and the guarantees offered by the module. At the inner layer, an implementation realizes the contract by concrete code, circuit structure, or state transition rules. A refinement map connects these layers by translating abstract obligations into lower-level conditions that can be checked mechanically. When the translation is sound, the implementation is not merely similar to the specification; it is a witness that the specification has been realized.

This layered view also clarifies reusability conditions. A module is reusable when its interface is stable under substitution, its contract is sufficiently local to be checked at the boundary, and its parameterization does not leak hidden dependencies into the surrounding system. In categorical terms, the module should behave functorially with respect to the chosen notion of context: replacing one compatible component by another should preserve the composite's well-formedness. In engineering terms, the same principle appears as encapsulation, dependency inversion, and interface-driven development.

The role of $\varepsilon(r)$ is especially important when exact boundary satisfaction is too strict to be informative at every scale. For example, a coarse model may admit a small but nonzero probability of mismatch at an interface, while a refined model reduces that mismatch by tightening the implementation or by increasing the resolution of the test harness. The metric therefore acts as a bridge between abstract contract satisfaction and empirical validation. It records how far the observed behavior departs from the declared boundary condition, and it provides a quantitative target for refinement.

Hypergraph categories and decorated cospans supply a particularly apt language for this bridge because they separate boundary structure from internal decoration. The boundary is what contracts talk about; the decoration is what refinement changes. Composition glues boundaries while combining decorations, so the formalism naturally supports modular reasoning. If each decorated component satisfies its local contract, then the composite can be assigned a derived contract whose validity follows from the compositional laws of the category. This is the mathematical expression of the engineering principle that local correctness should scale to system correctness.

The chapter's artifact requirement is therefore not incidental but constitutive of the exposition. A contract-checked library with CI tests makes the abstract claims inspectable. It should include at least one example of a coarse module and a refined module, a specification of the refinement

map between them, and a test suite that exercises the interfaces under composition. The continuous integration workflow should record whether the contract checks pass, whether any boundary violations are observed, and how those observations compare with the chosen $\varepsilon(r)$ threshold. Such a repository becomes a living proof object: the theory is encoded in the design, and the design is validated by the theory.

In summary, hierarchical modularity is the discipline of organizing systems so that interfaces, contracts, and refinement maps cooperate. The categorical viewpoint explains why this organization is compositional; the contract viewpoint explains how it is checkable; and the artifact viewpoint explains how it is made reproducible. The chapter develops these themes as a foundation for the later treatment of sequential composition, feedback, and traced structure, where the same emphasis on boundaries and invariants will reappear in a more dynamic setting.

3.2.2 Sequential Composition, Feedback, and Traces

This chapter studies the passage from combinational composition to stateful and cyclic composition. In the previous chapter, modules were connected through interfaces and refinement maps; here we add time, iteration, and feedback. The central question is how sequential composition behaves when outputs are fed back into inputs, and which algebraic structures correctly model that behavior without losing compositionality.

A useful starting point is the distinction between open and closed behavior. An open component exposes ports and can be composed with other components by wiring outputs to inputs. A closed component has no external interface and can be analyzed as a self-contained dynamical object. Sequential composition turns a family of open components into a larger open component, while feedback identifies selected outputs with selected inputs to form a loop. The resulting semantics must account for causality, delay, and the possibility of unstable or metastable behavior.

In hardware terms, feedback is not merely a wiring convenience. When a signal is routed back through a combinational path, the resulting loop may admit multiple fixed points, no fixed point, or transient oscillations. For this reason, the semantics of feedback must be guarded by delay, storage, or an explicit fixed-point principle. A common quantitative summary of this risk is the metastability probability $\varepsilon(r)$, viewed as a function of a relevant operating parameter r. In reliability analysis, one often reports a mean time between failures (MTBF) under feedback, emphasizing that the presence of a loop can change the failure profile even when the underlying combinational blocks are individually reliable.

The simplest disciplined way to introduce feedback is through state. A stateful component can be described as a transition system or as a machine with memory. Mealy machines produce outputs as a function of the current state and current input, while Moore machines produce outputs as a function of the current state alone. Both models support sequential composition: the output of one machine becomes the input of the next, and the internal state evolves step by step. Their difference matters in implementation, because Mealy-style outputs can react immediately to inputs, whereas Moore-style outputs are delayed by one state update. This distinction is often reflected in synthesis, verification, and timing closure.

Guarded recursion provides an abstract principle for defining such stateful and cyclic systems. A recursive specification is guarded when each recursive call is mediated by a constructor, delay, or other causal operator. Guardedness prevents ill-founded instantaneous self-reference and ensures that recursive equations can be interpreted as productive or contractive definitions. In categorical language, guarded recursion is closely related to traced structure and to fixed-point operators that are compatible with composition.

Traced categories supply a general algebraic framework for feedback. In a traced monoidal

category, one can feed an output wire back into an input wire while preserving the coherence laws of composition and tensoring. The trace abstracts the operation of closing a loop and is governed by axioms that express naturality, dinaturality, vanishing, superposing, and yanking. These laws ensure that feedback is not an ad hoc operation but a disciplined extension of sequential composition. In circuit semantics, the trace captures the idea that a loop can be moved, decomposed, or eliminated when the surrounding structure permits it.

The traced perspective also clarifies the relationship between circuits and automata. A finite-state automaton can be seen as a sequential machine whose transition structure is compatible with feedback through state. Conversely, many feedback circuits can be abstracted as automata by identifying a finite set of internal states and transition rules. This correspondence is especially useful when one wants to reason about equivalence: two implementations may differ syntactically while inducing the same input-output behavior over time.

A practical design question is how to realize memory elements from simpler gates. Flip-flops and latches can be built from NAND or NOR gates, and these constructions illustrate the boundary between combinational and sequential behavior. A latch is typically level-sensitive, while a flip-flop is edge-triggered or otherwise synchronized to a clock. The synchronous case is easier to analyze compositionally because state updates occur at designated times, which helps enforce a global notion of causality. Asynchronous closure is more delicate: without a shared clock or explicit delay discipline, feedback may create race conditions, hazards, or metastable states. The chapter therefore distinguishes synchronous closure, where composition respects a clocked update discipline, from asynchronous closure, where the semantics must explicitly manage timing and stabilization.

Automata minimization provides another lens on sequential composition. Given a machine, one may quotient its states by behavioral equivalence to obtain a smaller machine with the same observable behavior. This reduction preserves the symmetries that matter for external behavior while eliminating redundant internal distinctions. In the context of this book, minimization is not only an optimization technique but also a closure phenomenon: the class of behaviors represented by a machine is stable under quotienting by an appropriate equivalence relation. When symmetries are preserved, the minimized automaton retains the structural invariants relevant to composition and verification.

The equivalence problem becomes especially important when a design is implemented in hardware description language and then checked against a formal specification. The artifact of interest in this chapter is an HDL model paired with formal equivalence checking, typically mediated by an SMT-based backend. The workflow is straightforward in principle: write the sequential design in HDL, derive or encode the intended specification, and discharge the equivalence obligation with a solver. The solver checks that the implementation and specification agree on all admissible inputs and all reachable states, subject to the chosen abstraction. This artifact-oriented approach aligns with the book's broader claims-as-builds methodology: a semantic claim about feedback or state is accepted only when it is accompanied by a reproducible verification pipeline.

From a categorical standpoint, the main lesson is that sequential composition and feedback are not separate topics. Sequential composition builds larger systems from smaller ones; feedback closes systems into self-referential dynamics; traces provide the algebraic interface between the two. From a hardware standpoint, the main lesson is that state, timing, and equivalence must be treated together. A design is not fully understood until one knows how it composes, how it closes under feedback, how it behaves under minimization, and how its implementation can be checked against its specification.

The remainder of the chapter develops these ideas in a progression from machine models to traced semantics and then to verification artifacts. The intended outcome is a unified view in which sequential composition, feedback, and trace are different presentations of the same compositional

principle, constrained by causality and validated by formal equivalence.

Chapter focus. The source material for this chapter emphasizes four concrete threads: the metastability profile $\varepsilon(r)$ and MTBF under feedback; Mealy and Moore machines together with guarded recursion and traced categories; the construction of flip-flops and latches from NAND and NOR gates, including the distinction between synchronous and asynchronous closure; and automata minimization together with preserved symmetries. The chapter closes with an artifact-oriented workflow in which an HDL implementation is checked against a specification using formal equivalence, typically via SMT.

Reading guide. Readers should interpret the later sections of this chapter as progressively refining the same theme. First, sequential composition is treated as ordinary wiring of stateful components. Next, feedback is formalized as a trace or guarded recursion. Then the hardware realization of memory elements is used to connect the abstract semantics to clocked and unclocked circuits. Finally, equivalence checking turns the semantic account into a reproducible verification artifact.

Outlook. These constructions prepare the transition to the next chapter, where constructive logic provides a different but compatible language for digital composition. There, the emphasis shifts from state and feedback to logical structure, but the same compositional discipline remains in force.

3.2.3 Constructive Logic: Heyting Semantics for Digital Composition

Constructive logic provides a semantics for digital composition in which proofs are not merely external justifications but internal witnesses of implementability. In this chapter, we interpret circuit specifications constructively, emphasizing the distinction between what can be proved to exist and what can be exhibited as an explicit artifact. This perspective is especially natural for digital design, where a specification should ideally correspond to a realizable implementation and where failure modes can be analyzed as counterexamples rather than as abstract negations.

A useful operational summary is given by the quantity $\varepsilon(r)$, which we interpret as a comparison between proof obligations and counterexample rate. Informally, a low value of $\varepsilon(r)$ indicates that the design space is dominated by obligations that can be discharged constructively, while a high value indicates that counterexamples are frequent relative to the claims being made. In a build-oriented workflow, $\varepsilon(r)$ can be used as a chapter-specific diagnostic: it does not replace correctness, but it helps track whether a proposed specification is becoming more amenable to constructive realization.

The semantic setting for this chapter is provided by Heyting algebras rather than Boolean algebras. A Boolean algebra validates classical principles such as excluded middle in full generality, whereas a Heyting algebra models intuitionistic logic, where negation is understood as implication into falsity and where double negation need not collapse to the original proposition. For digital composition, this distinction matters because many design claims are naturally constructive: one may want an explicit witness for a signal assignment, a decomposition, or a refinement map, rather than a nonconstructive existence argument.

In this setting, logical connectives correspond to algebraic operations that preserve constructive meaning. Conjunction models simultaneous satisfaction of specifications, disjunction models a choice between alternatives, and implication models refinement or transformation of one specification into another. Closure under constructive connectives is therefore a central theme: if a

family of circuit specifications is closed under the relevant intuitionistic operations, then composite designs can be reasoned about internally without leaving the constructive framework. By contrast, classical reasoning via unrestricted double negation can obscure implementation content. The double-negation caveat is that a statement of the form $\neg\neg P$ does not, in general, provide a witness for P; in circuit terms, a proof that a behavior cannot be refuted is not the same as a construction of the behavior itself.

The contrast between Heyting and Boolean semantics becomes especially visible in the treatment of exclusive-or. Classically, XOR is often introduced as a derived connective with a simple truth table. Constructively, however, one must be careful about how the exclusive-or specification is presented, because the intended meaning is not merely that exactly one of two propositions holds in an abstract sense, but that one can determine which side holds and provide the corresponding witness. This makes constructive XOR a useful test case for the difference between Boolean semantics and Heyting semantics in digital composition.

The Curry–Howard correspondence supplies the bridge from logic to circuits. Under this interpretation, types are specifications and terms are implementations. A proposition corresponds to a type, a proof corresponds to a program, and a proof term can be read as a circuit or circuit fragment that realizes the specification. This viewpoint is particularly effective for compositional hardware design: if a type expresses the interface contract of a module, then a term inhabiting that type is an executable witness that the module satisfies its contract. Composition of proofs becomes composition of circuits, and logical elimination rules become wiring principles.

From this perspective, constructive logic is not merely a philosophical preference; it is a discipline for artifact generation. A proof that a circuit exists should ideally be accompanied by the circuit itself, or at least by a machine-checkable derivation from which the circuit can be extracted. This is the motivation behind the use of proof assistants such as Agda and Coq. In such systems, one can formalize a specification, prove that it is inhabited, and then extract executable code or circuit descriptions from the proof term. The resulting artifact is not an informal sketch but a buildable object that can be checked, compiled, and integrated into a larger design flow.

The artifact-oriented workflow suggested here has three layers. First, one states a constructive specification in a type-theoretic or intuitionistic logical language. Second, one proves the specification using only constructive principles, thereby ensuring that the proof carries computational content. Third, one extracts or translates the proof into an implementation, such as a circuit netlist, a hardware description, or a verified functional model. In this way, the logical semantics and the engineering semantics remain aligned.

A practical consequence of this alignment is that the semantics of failure also becomes constructive. When a specification is not realizable, the relevant information is often a counterexample, a refutation, or a witness of inconsistency in the assumed constraints. Thus the chapter's emphasis on $\varepsilon(r)$ can be read as a way of balancing proof obligations against counterexample rate: the more the design is expressed constructively, the more directly one can distinguish between a missing proof and a genuine impossibility.

The chapter therefore sets up a semantic framework for later constructions in the book. It complements the earlier algebraic and compositional foundations by showing how logical structure can be used to certify digital artifacts, and it prepares the ground for subsequent case studies in which gates, modules, and larger circuits are derived from specifications rather than assembled ad hoc. The central message is that constructive logic, Heyting semantics, and Curry–Howard interpretation together provide a principled route from specification to implementation, with proofs serving as build scripts and extracted circuits serving as executable evidence.

3.2.4 Case Study—From NAND to XOR to Half-Adder

This chapter is a worked case study in closure under composition: starting from a single primitive gate, NAND, we derive the familiar basis $\neg$, $\wedge$, and $\vee$, then build XOR and finally a half-adder. The point is not merely that these constructions are possible, but that the resulting implementations carry observable artifacts—depth, size, and timing behavior—that matter when the same Boolean specification is realized in hardware.

We will keep two views in play throughout. At the specification level, a circuit denotes a Boolean function and may exhibit invariances such as commutativity under swapping inputs. At the implementation level, the same function can be realized by many structurally different netlists, each with different depth, gate count, fanout structure, and sensitivity to timing-window violations. In this sense, the chapter illustrates how semantic equivalence can coexist with distinct physical behavior.

A useful error metric for this setting is the gate-level timing-window violation rate, denoted $\varepsilon(r)$. Informally, $\varepsilon(r)$ measures the fraction of runs in which a circuit exhibits a hazard or incorrect transient behavior under a given fault or timing regime r. We will refer to $\varepsilon(r)$ when discussing hazards and fault sensitivity, especially for derived implementations whose logical correctness does not by itself guarantee robust behavior.

Starting from NAND, one obtains negation by tying the inputs together:

$$\neg a \equiv a \uparrow a,$$

where $\uparrow$ denotes NAND. From this, conjunction and disjunction can be expressed in the usual way:

$$a \wedge b \equiv \neg(a \uparrow b), \qquad a \vee b \equiv (\neg a) \uparrow (\neg b).$$

These identities show that NAND is functionally complete. They also expose a basic depth/size trade-off: a direct specification may be compact as a formula, while a NAND-only implementation expands into multiple primitive gates. Different decompositions can reduce depth at the cost of additional gates, or reduce size at the cost of longer critical paths.

The XOR gate is a particularly instructive example because it admits several equivalent forms and is sensitive to implementation choice. One standard decomposition is

$$a \oplus b \equiv (a \wedge \neg b) \vee (\neg a \wedge b).$$

This expression makes the symmetry of XOR explicit: swapping a and b leaves the function unchanged. Other algebraically equivalent forms are often preferable in hardware synthesis, depending on the available primitives and optimization objective. For example, XOR can also be written using NAND-only subcircuits by substituting the derived forms of $\neg$, $\wedge$, and $\vee$, though the resulting netlist may differ substantially in depth and hazard profile. In practice, one may choose between a compact but deeper realization and a flatter realization with more shared structure, depending on the target timing constraints.

The half-adder combines XOR and AND to produce a sum bit and a carry bit. Its specification is

$$\mathrm{sum}(a, b) = a \oplus b, \qquad \mathrm{carry}(a, b) = a \wedge b.$$

As a Boolean relation, the half-adder is invariant under swapping the inputs a and b: both outputs are unchanged by the transposition $(a, b) \mapsto (b, a)$. This invariance is a semantic property of the specification, not of any particular gate-level realization. A NAND-based implementation may preserve the same input symmetry only up to circuit isomorphism; the physical netlist can still privilege one input path over another, affecting delay and transient behavior.

This distinction between specification and artifact is central to the case study. Two implementations can compute the same half-adder truth table while differing in gate count, logic depth, reconvergent fanout, and susceptibility to hazards. In particular, a design that is logically correct may still exhibit a larger $\varepsilon(r)$ under a fault model that perturbs arrival times or introduces brief glitches. Thus, correctness at the Boolean level is necessary but not sufficient for robust implementation.

A compact NAND-only realization of the half-adder can be obtained by first building $\neg$, $\wedge$, and $\vee$, then assembling XOR and carry from those derived blocks. This modular route is easy to reason about and aligns with the compositional perspective developed earlier in the book. However, synthesis tools may choose alternative factorizations that reduce depth or exploit shared subexpressions. The resulting artifact is therefore best understood as a point in a design space rather than as a unique canonical form.

For reproducibility, it is useful to pair the logical derivation with an executable artifact. A minimal Verilog description can encode the NAND-derived half-adder, and a testbench can enumerate all four input combinations to confirm the truth table. The same harness can be extended with fault injection or timing perturbations to estimate $\varepsilon(r)$ under specified conditions. In that setting, the testbench serves not only as a correctness check but also as a measurement instrument for implementation artifacts.

```
module half_adder_nand (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);
    wire n1, n2, n3, n4, n5;

    nand (n1, a, b);
    nand (n2, a, n1);
    nand (n3, b, n1);
    nand (sum, n2, n3);
    nand (carry, n1, n1);
endmodule

module half_adder_nand_tb;
    reg a, b;
    wire sum, carry;
    integer i;

    half_adder_nand dut (.a(a), .b(b), .sum(sum), .carry(carry));

    initial begin
        for (i = 0; i < 4; i = i + 1) begin
            {a, b} = i[1:0];
            #1;
            $display("a=%b b=%b | sum=%b carry=%b", a, b, sum, carry);
        end
        $finish;
```

```
    end
endmodule
```

The chapter's working conclusion is that NAND closure gives a complete algebraic basis for Boolean construction, but the path from basis to system is not neutral. Each derivation choice affects depth, size, symmetry realization, and hazard behavior. The half-adder is therefore a small but representative example of the book's broader theme: compositional equivalence at the semantic level does not erase the structure of the implementation that realizes it.

In summary, the case study provides a concrete bridge from the abstract closure results of the earlier chapters to the implementation-sensitive concerns that recur later in the book. The same Boolean function can be presented as an equation, a netlist, or a testable artifact, and each presentation reveals different invariants and different failure modes. That multiplicity is not a defect; it is the central phenomenon the book aims to formalize.

3.3 Part III: Symmetries and Applications

3.3.1 Symmetry Groups of Boolean Functions and Circuits

This chapter studies symmetry as an organizing principle for Boolean functions and circuits. The central objects are actions of the symmetric group Σ_n on n-variable Boolean functions, their automorphism groups, and the induced orbit–stabilizer structure that partitions functions into symmetry classes. These notions provide a bridge between abstract equivalence relations on truth tables and concrete transformations of circuit descriptions.

Let $f : \mathbb{B}^n \to \mathbb{B}$ be a Boolean function. A permutation $\pi \in \Sigma_n$ acts on inputs by reindexing coordinates, and we write f^π for the transported function obtained by permuting variables according to π. The automorphism group of f is the subgroup

$$\mathrm{Aut}(f) = \{\pi \in \Sigma_n : f^\pi = f\},$$

which measures the internal symmetries of the function. The orbit of f under Σ_n consists of all functions obtainable by permuting variables, while the stabilizer records the permutations that leave the function unchanged. In circuit settings, the same ideas apply to the input interface of a circuit, and symmetry can be used to simplify search, canonicalization, and equivalence checking.

A finer notion of equivalence is NPN equivalence, which combines negation of inputs, permutation of inputs, and negation of the output. Two Boolean functions are NPN-equivalent if one can be transformed into the other by these operations. This equivalence relation is especially useful in classification problems, because it identifies functions that differ only by trivial relabelings and polarity changes. Canonical forms provide a representative for each NPN class, enabling systematic classification and comparison. In practice, a canonicalizer attempts to map each function to a unique normal form, while preserving the equivalence class under the chosen transformation group.

For circuit analysis, NPN equivalence is often paired with structural invariants extracted from the truth table or from spectral data. A canonical form can serve as a compact fingerprint for a function or subcircuit, supporting database indexing, duplicate detection, and benchmark generation. The chapter's artifact-oriented perspective treats canonicalization not only as a theoretical classification tool but also as an executable component in a symmetry analysis pipeline.

Beyond permutation symmetries, Boolean functions admit linear and affine symmetries. The general linear group $\mathrm{GL}(n, 2)$ acts on $\mathbb{B}^n$ by invertible linear transformations over $\mathbb{F}_2$, and the affine general linear group $\mathrm{AGL}(n, 2)$ extends this action by translations. These groups are relevant when one studies symmetries that preserve algebraic structure rather than merely variable names. Linear

and affine equivalence are coarser than permutation equivalence and are especially important in contexts such as cryptography, coding theory, and reversible or quantum-flavored computation.

Spectral methods provide a complementary approach to symmetry analysis. Using the Walsh–Hadamard transform, a Boolean function can be represented by its spectral coefficients, which encode correlations with parity functions. These coefficients often reveal hidden regularities that are not obvious from the truth table alone. In particular, approximate symmetry can be quantified by a spectral correlation measure $\varepsilon(r)$, where r indexes a candidate symmetry or transformation and the value of $\varepsilon(r)$ indicates how closely the function aligns with that symmetry. Such measures are useful when exact invariance fails but near-symmetry still carries algorithmic or structural significance.

The spectral viewpoint is especially effective for detecting approximate automorphisms, ranking candidate permutations, and guiding search over large symmetry groups. In a workflow for symmetry discovery, one may first compute spectral signatures, then restrict attention to transformations that preserve or nearly preserve those signatures, and finally verify exact equivalence or automorphism conditions by direct evaluation. This combination of coarse spectral filtering and exact checking is often more efficient than exhaustive enumeration.

The chapter also emphasizes the relationship between symmetry and classification. Functions and circuits can be organized into equivalence classes under Σ_n-actions, NPN transformations, or affine transformations, depending on the level of abstraction required. Each choice of group action induces a different notion of canonical representative and a different notion of complexity for the classification problem. The resulting hierarchy of symmetry notions mirrors the broader theme of the book: closure properties and compositional structure become more tractable when the relevant invariances are made explicit.

An important practical outcome is the construction of a symmetry oracle and canonicalizer with benchmarks. A symmetry oracle is a procedure that, given a Boolean function or circuit, returns information about its automorphism group, candidate symmetries, or equivalence class membership. A canonicalizer uses this information to produce a normal form. Benchmarks then evaluate correctness, speed, and robustness across families of functions with varying degrees of symmetry. Such artifacts make the chapter's ideas reproducible and testable, and they provide a concrete bridge from theory to implementation.

In summary, the chapter develops three intertwined perspectives on symmetry in Boolean settings: group actions and automorphisms, NPN and affine equivalence for classification, and spectral methods for exact and approximate symmetry detection. Together these tools support both mathematical understanding and practical algorithms for analyzing Boolean functions and circuits.

3.3.2 Reliability, Noise, and Robust Closure

This chapter studies how computation behaves when gates, wires, or intermediate states are subject to perturbation. The central quantity is a reliability function $\varepsilon(r)$, interpreted as the probability of correct operation as a function of a resource parameter r, such as depth, size, code distance, or redundancy level. In the simplest form, one may write

$$\varepsilon(r) = 1 - \Pr[\text{error}].$$

The point of the chapter is not merely to measure failure, but to understand how compositional structure can be made robust under noise, and when redundancy can be organized so that reliability improves rather than degrades as systems scale.

A noisy computation model treats elementary operations as imperfect. A gate may flip its intended output with some small probability, a wire may corrupt a signal, and a composed circuit

may accumulate error across layers. In this setting, the design problem becomes one of robust closure: given a family of operations closed under composition in the idealized setting, how can one construct a corresponding family of fault-tolerant implementations whose effective behavior remains within the same computational class? The answer is typically not to eliminate noise entirely, but to encode information and computation so that local faults are detected, corrected, or rendered harmless by aggregation.

A classical strategy is redundancy. If a bit is represented by several physical bits, then a decoding rule can recover the intended logical value even when some fraction of the physical bits are corrupted. The simplest example is majority logic: if three copies of a bit are transmitted or stored, then the majority vote returns the correct value whenever at most one copy is faulty. More generally, majority-based schemes can be iterated hierarchically, yielding recursive constructions in which each level of encoding suppresses the effective error rate of the level below. Such schemes are closely related to von Neumann-style reliable computation, in which unreliable components are assembled into a more reliable whole by combining redundancy with local correction.

Von Neumann schemes are especially important because they show that reliable computation is not limited to perfectly reliable primitives. Instead, one can build a threshold phenomenon: if the physical error rate is below a certain bound, then sufficiently deep or sufficiently redundant constructions can drive the logical error rate arbitrarily low. This threshold behavior is one of the chapter's central themes. It provides a quantitative closure principle: below threshold, the class of computable functions is stable under noisy implementation, in the sense that the intended logical operation can be recovered with high probability after encoding, composition, and decoding.

Error-correcting codes provide a more systematic language for this phenomenon. A code is not only a storage device but also a compositional symmetry: it identifies a structured subset of states together with encoding and decoding maps that preserve logical information under a controlled family of perturbations. From this perspective, the code space is an invariant substructure, and the correction procedure is a projection back onto that invariant. The algebraic viewpoint is useful because it emphasizes that fault tolerance is not an ad hoc patch; it is a design principle based on symmetry, invariance, and controlled quotienting of noisy degrees of freedom.

This perspective also clarifies why codes compose. If one encoding protects against a certain class of local errors and a second encoding protects the encoded representation against higher-level faults, then the combined system can inherit robustness across multiple scales. Such nested constructions are the reliability analogue of hierarchical modularity: each layer abstracts away the details of the layer below while preserving the logical content. In practice, this means that the same conceptual tools used earlier in the book to study compositional closure can be applied to fault tolerance, with the additional requirement that the relevant equivalences be probabilistic rather than exact.

Threshold theorems formalize this intuition. They assert, under suitable assumptions on the noise model and the available correction gadgets, that there exists a constant p_* such that if the physical error rate $p < p_*$, then arbitrarily long computations can be performed with arbitrarily small logical error by increasing redundancy and using recursive correction. The exact value of the threshold depends on the architecture, the noise model, and the correction protocol, but the qualitative conclusion is robust: reliable computation is possible in principle even when every component is imperfect, provided the imperfections are sufficiently sparse and sufficiently well-behaved.

The chapter is also concerned with how reliability scales with resource usage. The function $\varepsilon(r)$ is intended to capture this dependence, whether r denotes circuit depth, code distance, number of redundant copies, or another measure of implementation scale. In a favorable regime, $\varepsilon(r)$ increases toward one as r grows, reflecting improved reliability. In an unfavorable regime, additional depth

may simply accumulate more opportunities for failure. The design challenge is therefore to choose encodings and correction mechanisms so that the gain from redundancy dominates the cost of extra operations.

A practical artifact for this chapter is a fault-injection harness. Such a harness perturbs a circuit or abstract computation according to a specified noise model, runs repeated trials, and records empirical error rates as a function of redundancy parameters. The resulting error-versus-redundancy plots provide an operational view of threshold behavior: one can observe whether additional copies, larger code distance, or deeper correction layers reduce the logical error rate, and at what point diminishing returns set in. These plots are not merely illustrative; they are part of the chapter's reproducibility protocol and serve as a bridge between theoretical claims and empirical validation.

Taken together, noisy gates, majority logic, error-correcting codes, and threshold theorems show that closure in digital systems must sometimes be understood probabilistically. The ideal algebra of composition remains relevant, but it is supplemented by a robustness calculus that tracks how errors propagate, how they are suppressed, and when they can be made asymptotically negligible. This chapter develops that viewpoint as a foundation for later discussions of reliable architectures, fault-tolerant program semantics, and symmetry-aware design under uncertainty.

3.3.3 Algebra of Programs & Hardware DSLs

This chapter connects algebraic views of programs with the design of hardware-oriented domain-specific languages (DSLs). The central theme is that both programs and circuits can be treated as elements of an algebraic theory: they are built from generators, constrained by equations, and manipulated by rewriting rules that preserve meaning. In this setting, equational reasoning is not merely a proof technique; it is also a practical compilation strategy and a design principle for executable artifacts.

A useful starting point is the notion of a law coverage metric, written here as $\varepsilon(r)$, which measures property coverage or mutation score for a proposed set of laws. Informally, $\varepsilon(r)$ asks how well a collection of equations detects incorrect variants of a specification or implementation. In a DSL workflow, this metric can be used to compare candidate rewrite systems, to assess whether a normal form is sufficiently discriminating, and to guide the selection of laws that support both optimization and verification. The point is not that a single scalar score captures semantic adequacy, but that law coverage provides a reproducible way to evaluate whether the algebraic presentation of a language is doing real work.

The algebraic semantics of programs is naturally expressed using Lawvere theories. A Lawvere theory presents operations and equations in a way that is especially well suited to algebraic data types, effectful computations, and compositional program structure. In the presence of effects, one often studies how the theory is enriched, extended, or combined so that operations such as state, choice, exceptions, or interaction can be represented without losing the equational discipline. This perspective also clarifies the relation to operads and PROPs. Operads organize multi-input operations with substitution, while PROPs extend the picture to operations with multiple inputs and multiple outputs, which is particularly relevant for circuits and signal-flow formalisms. Thus, Lawvere theories, operads, and PROPs can be viewed as complementary algebraic languages for specifying compositional systems at different levels of arity and wiring structure.

For hardware DSLs, the design problem is to choose combinators that are expressive enough to describe circuits while still admitting a tractable equational theory. A well-designed embedded DSL (EDSL) for circuits typically exposes a small set of primitive combinators for composition, fanout, pairing, and basic gates, then relies on derived combinators and rewrite rules to express

higher-level structure. One goal is to identify normal forms: canonical representatives of circuit expressions that make equivalence checking and optimization easier. Normal forms are valuable because they turn semantic equality into syntactic comparison after normalization, provided the rewrite system is sound and sufficiently complete for the intended fragment.

Rewriting systems supply the operational mechanism for this approach. A rewrite system consists of oriented equations used to transform expressions into simpler or more canonical ones. Two properties are especially important. First, termination ensures that rewriting does not continue indefinitely; every expression eventually reaches a normal form. Second, confluence ensures that the final normal form does not depend on the order in which rewrite rules are applied. When both properties hold, rewriting becomes a robust decision procedure for equality modulo the chosen laws. In practice, one often seeks a weaker but still useful notion of completeness: the rewrite system should be complete with respect to the intended semantics, meaning that semantic equivalence implies convertibility by the rules, at least within the chosen fragment of the language.

These ideas are particularly effective when the DSL is intended to support equational reasoning as a first-class feature. In such a language, users can state laws about associativity, commutativity, distributivity, identities, annihilators, or circuit-specific identities, and then rely on the system to normalize expressions or prove equivalences. The artifact-oriented goal is to make these laws executable: the same equations that appear in documentation or proofs should also drive simplification, testing, and compilation. This is where law coverage $\varepsilon(r)$ becomes operationally meaningful, because a rewrite set that is elegant on paper but weak in mutation score may fail to distinguish important semantic cases.

A practical chapter artifact is therefore a DSL equipped with equational reasoning and property-based testing, for example via QuickCheck-style randomized testing. Such an artifact can expose a small core language of circuit combinators, a library of rewrite rules, a normalization procedure, and a test harness that checks laws against randomly generated expressions or circuit instances. The testing layer does not replace proof, but it provides a reproducible sanity check for the rewrite theory and helps detect missing laws, unsound orientations, or incomplete normal forms. In the broader workflow of this book, the artifact serves as a bridge between abstract algebraic semantics and buildable hardware/software tools.

The overall message is that algebraic program semantics and hardware DSL design are two sides of the same compositional coin. Lawvere theories provide a semantic vocabulary, operads and PROPs organize the wiring discipline, rewriting systems implement normalization and equivalence checking, and property-based testing supplies empirical feedback on the adequacy of the chosen laws. Together, these ingredients support a disciplined approach to DSL construction in which equations are not only descriptive but also computationally effective.

In a minimal implementation sketch, one can separate the language into three layers. The first layer defines the syntax of expressions, including generators and combinators. The second layer defines the equational theory, either as a set of rewrite rules or as a normalization function that orients those rules. The third layer defines the validation harness, which generates random terms, checks that the laws hold, and estimates $\varepsilon(r)$ for the current rule set. This separation makes it easier to evolve the DSL without conflating syntax, semantics, and testing.

A useful design criterion is that the normal form should be stable under the intended algebraic laws and informative enough to support downstream tasks such as optimization, equivalence checking, and code generation. If the normal form is too coarse, distinct behaviors may collapse; if it is too fine, the rewrite system may become brittle or incomplete. The balance between expressiveness and canonicalization is therefore central to the chapter's perspective on hardware DSLs.

From the standpoint of the larger book, this chapter also prepares the ground for later discussions of dataflow graphs, parallel symmetries, and reversible or quantum-flavored formalisms. The

same algebraic discipline that supports circuit DSLs can be extended to richer wiring structures, more constrained resource models, and more sophisticated notions of equivalence. The present chapter isolates the core pattern: specify generators, state laws, orient them into a terminating and confluent rewrite system when possible, and validate the resulting theory with executable tests.

Artifact sketch. A representative artifact for this chapter is a small DSL for circuits with the following components: a typed expression datatype; primitive combinators for identity, composition, tensoring or pairing, and selected gates; a rewrite engine implementing the chosen equations; a normalization routine returning canonical forms; and a QuickCheck-style suite that checks the laws on randomly generated terms. The artifact should report both pass/fail results for individual laws and an aggregate coverage estimate $\varepsilon(r)$, so that changes to the law set can be evaluated quantitatively.

Takeaway. Algebraic semantics is not an abstract overlay on implementation; it is a way to make implementation more reliable, more compositional, and more testable. When a DSL is designed around equations that are sound, terminating where possible, and empirically well covered, the resulting system supports both formal reasoning and practical engineering.

3.3.4 Dataflow, Graphs, and Parallel Symmetries

This chapter examines how streaming computation, graph structure, and symmetry interact in parallel systems. The central concern is not only whether a computation can be expressed as a dataflow graph, but whether that graph admits compositional scheduling, symmetry-aware optimization, and implementation choices that preserve throughput under resource constraints. In this setting, performance is summarized by an error-and-efficiency profile such as $\varepsilon(r)$, where the relevant observables include throughput, latency, and deadline miss rate.

A useful starting point is the class of Kahn networks, in which processes communicate through unbounded FIFO channels and the overall behavior is determined compositionally by the local process semantics. Such networks provide a clean model for streaming systems because they separate functional behavior from scheduling policy. In practice, however, the semantics of a streaming pipeline must be paired with a compositional scheduling discipline: the order in which actors fire, the allocation of buffers, and the handling of backpressure all affect whether the implementation meets its timing targets. Streaming operads offer one way to formalize this compositionality, treating pipeline fragments as operations that can be assembled while preserving typing and interface structure.

Graph structure becomes especially important when the pipeline exhibits nontrivial automorphisms. If a dataflow graph has symmetries, then equivalent subcomputations may be permuted without changing the observable behavior. These graph automorphisms can be exploited to obtain load-balanced symmetries: work can be distributed across processors or accelerators in a way that respects the graph's invariant structure while reducing imbalance. In favorable cases, symmetry-aware scheduling identifies interchangeable regions of the graph and uses them to improve utilization without changing the denotation of the computation.

The optimization laws in this chapter are expressed through fusion and fission. Fusion combines adjacent stages to reduce communication overhead and improve locality, while fission splits a stage into parallel copies to increase throughput or match hardware granularity. These transformations are not merely heuristic; they are constrained by the semantics of the dataflow graph, by buffer capacities, and by the need to preserve the relevant performance metrics. On GPUs and FPGAs, such laws often appear as tiling decisions: a streaming kernel may be partitioned into tiles that

fit shared memory, register files, or on-chip resources, and the resulting tiling must be compatible with the graph's dependency structure.

A compact way to state the chapter's design principle is that symmetry should be treated as an optimization resource. When a pipeline admits automorphisms, those symmetries can guide both scheduling and placement, yielding implementations that are not only correct but also balanced across available hardware. Conversely, when fusion or fission breaks an apparent symmetry, the resulting asymmetry must be justified by a measurable gain in $\varepsilon(r)$, such as improved throughput, reduced latency, or a lower deadline miss rate.

The artifact associated with this chapter is a family of streaming kernels equipped with symmetry transforms. The intended use is to demonstrate how a kernel can be rewritten by graph automorphisms, fused or fissioned according to resource constraints, and then mapped to a parallel target such as a GPU or FPGA. The artifact should record the original graph, the transformed graph, the scheduling assumptions, and the measured values of throughput, latency, and deadline miss rate so that the optimization can be reproduced and compared across implementations.

Taken together, these ideas support a symmetry-aware view of parallel optimization: the graph provides the compositional skeleton, the scheduling discipline determines executable behavior, and the symmetry structure identifies legal transformations that can improve performance. The chapter's guiding principle is that parallel efficiency is not an external afterthought but a property that can be tracked, transformed, and validated within the same compositional framework as the computation itself.

3.3.5 Reversible & Quantum-Flavored Hierarchies

This chapter develops a reversible and quantum-flavored view of compositional hierarchies, emphasizing how information-preserving structure changes the algebra of design. The central theme is that once a computation is required to be reversible, or even merely approximately reversible in a quantum sense, the familiar closure properties of ordinary Boolean hierarchies must be refined by resource accounting, linear structure, and rewrite discipline.

A useful organizing quantity is an error or cost functional $\varepsilon(r)$, where r denotes a candidate realization, refinement, or rewrite path. In the reversible setting, $\varepsilon(r)$ may be read as a reversible cost: the amount of auxiliary structure, cleanup, or overhead required to implement a map without losing information. In the quantum setting, the same symbol can be interpreted as a quantum error measure, for example a diamond-norm deviation between an intended channel and its implementation. The point is not that these notions are identical, but that both serve as closure-sensitive metrics: they quantify how far a construction is from living inside the desired symmetry class.

Reversible computation is the first major example. A map is reversible when it is bijective on the relevant state space, so that no information is discarded. Classical reversible logic is typically generated by gates such as Toffoli and Fredkin, which are universal for reversible computation in the appropriate sense. These gates illustrate a general principle: universality under reversibility is not obtained by arbitrary fan-out and erasure, but by controlled permutations of state. When a computation produces temporary garbage, Bennett cleanup provides a standard method for restoring reversibility. The idea is to compute the desired output, copy it in a reversible manner when possible, run the computation backward to erase the workspace, and thereby recover a clean output together with the original input structure. This cleanup pattern is a canonical artifact of reversible synthesis, and it makes explicit that reversibility is often achieved by adding structured bookkeeping rather than by changing the logical specification.

The reversible viewpoint naturally leads to linear and affine structure over $\mathbb{F}_2$. In ordinary Boolean computation, duplication and deletion are freely available structural rules. In contrast, re-

versible and quantum-flavored settings restrict these rules, and the resulting algebra is often better described by linear or affine maps over the two-element field. Parity becomes a fundamental invariant: many reversible transformations preserve parity symmetries, and these symmetries constrain what can be built without ancillary resources. From this perspective, affine transformations over $\mathbb{F}_2$ form a tractable subhierarchy in which composition is closed, but only under the discipline that outputs are assembled from linear combinations and constant shifts compatible with the ambient symmetry.

Quantum computation sharpens these constraints further. Quantum universality is commonly expressed through gate sets such as Clifford plus T, where Clifford operations provide a stabilizer-preserving backbone and the non-Clifford T gate supplies the missing expressive power. The resulting hierarchy is not merely a larger gate set; it is a structured decomposition of computation into symmetry-preserving and symmetry-breaking components. This decomposition is especially useful when reasoning about approximation, synthesis cost, and fault-tolerant compilation. The chapter therefore treats quantum universality as a closure problem under a richer notion of equivalence, one in which exact equality is replaced by channel approximation and resource overhead.

Categorical formalisms provide a compact language for these ideas. In particular, the ZX-calculus can be viewed as a PROP, that is, a symmetric monoidal category whose objects are natural numbers and whose morphisms encode string-diagrammatic quantum processes. Within this setting, generators and rewrite rules capture the algebra of quantum circuits in a form amenable to equational reasoning. The PROP perspective is important because it makes compositionality explicit: sequential and parallel composition are built into the syntax, while rewrite rules express the admissible symmetries of the theory. As a result, ZX-calculus serves both as a semantic framework and as a practical rewriting system for circuit optimization and verification.

The artifact-oriented aspect of the chapter is reversible synthesis and ZX rewriting via a prover. In the reversible case, synthesis aims to construct a circuit from a specification while minimizing ancilla use, garbage, and cleanup overhead. In the quantum-flavored case, the goal is to derive a circuit or diagrammatic proof by applying sound ZX rewrites, ideally with machine-checked support. A prover can serve as the executable witness that a proposed rewrite sequence preserves semantics, and it can also track the cost functional $\varepsilon(r)$ across alternative derivations. This makes the chapter's theme concrete: closure is not only a theorem about what can be expressed, but also a build process in which reversible and quantum constraints are enforced by explicit artifacts.

Taken together, these topics show that reversible and quantum-flavored hierarchies are best understood as symmetry-constrained extensions of the compositional framework developed earlier in the book. Reversibility replaces erasure with cleanup, linearity replaces arbitrary copying with parity-aware structure, and quantum formalisms replace exact Boolean equality with diagrammatic and metric equivalence. The resulting hierarchy is richer, but also more disciplined: every construction must account for information flow, symmetry preservation, and the cost of leaving the ideal subcategory of perfectly reversible or perfectly unitary behavior.

For later use, it is helpful to isolate the following working distinctions. First, reversible cost measures the overhead of embedding a computation into a bijective or information-preserving form. Second, quantum error measures the deviation of an implemented process from an ideal channel, with the diamond norm serving as a standard operational metric. Third, linear and affine structure over $\mathbb{F}_2$ captures the algebraic residue left after copying and deletion are restricted. Fourth, the ZX-calculus provides a diagrammatic PROP in which these constraints can be expressed as rewrite rules rather than as ad hoc circuit identities. These distinctions will recur whenever the book compares exact closure, approximate closure, and closure under resource-bounded synthesis.

A compact way to summarize the chapter is to view it as a ladder of increasingly constrained closure notions:

- reversible closure, where maps must be bijective and cleanup is part of the implementation;
- affine closure over $\mathbb{F}_2$, where parity and linear structure replace unrestricted duplication;
- quantum closure, where exact equality is replaced by approximation in a metric such as the diamond norm;
- diagrammatic closure, where sound rewrites in a PROP such as ZX-calculus provide the proof theory of the hierarchy.

This ladder is not meant to separate the topics into unrelated cases. Rather, it exhibits a common pattern: each additional constraint narrows the admissible morphisms, but also supplies a more refined language for synthesis, verification, and optimization.

In practical terms, the chapter's build artifact is a reversible synthesis and ZX-rewrite workflow. A specification is first normalized into a form suitable for reversible embedding or diagrammatic translation. Ancilla allocation, garbage management, and Bennett-style cleanup are then recorded as explicit steps in the reversible case. In the quantum-flavored case, the corresponding artifact is a rewrite trace in which each transformation is justified by a prover and annotated by its effect on $\varepsilon(r)$. The resulting trace is not merely a proof object; it is also a reproducible optimization log.

This perspective aligns the chapter with the broader book-level protocol: claims are treated as builds, and closure statements are treated as executable invariants. Here, the invariant is not ordinary Boolean closure alone, but closure under reversibility, linearity, and diagrammatic quantum rewriting. The chapter therefore serves as a bridge between classical compositional hierarchies and the more delicate resource-sensitive hierarchies that arise in reversible and quantum computation.

3.3.6 Cryptographic Constructions & Symmetry Constraints

This chapter examines how symmetry constraints shape both the design and the analysis of cryptographic primitives. The central theme is that useful cryptographic behavior often requires a careful balance: enough structure to support efficient implementation and provable reasoning, but enough asymmetry to resist unintended invariants, equivalences, and attacks. In this setting, symmetry is not merely an aesthetic property; it is a design parameter that can strengthen or weaken security depending on how it interacts with composition.

A recurring quantitative lens is the round-dependent error profile $\varepsilon(r)$, where r denotes the number of rounds in a construction. In practice, $\varepsilon(r)$ may summarize how quickly avalanche behavior emerges, or how differential and linear biases decay as rounds are added. From the standpoint of closure and compositional hierarchies, the key question is whether the intended mixing behavior is preserved under iteration, or whether hidden invariants survive round composition and create exploitable regularities.

For substitution–permutation networks and related designs, the S-box is the primary local source of nonlinearity. Its symmetry profile is therefore critical. Two S-boxes related by affine equivalence or CCZ equivalence may share important cryptanalytic properties, even when their concrete representations differ. In particular, nonlinearity and differential uniformity are standard measures used to assess resistance to linear and differential attacks, and they also serve as coarse invariants under many symmetry-preserving transformations. The design problem is not simply to maximize these metrics in isolation, but to understand which equivalence classes preserve them and which transformations alter the effective attack surface.

Round composition introduces a second layer of structure. In SPN and Feistel constructions, the interaction between local nonlinear layers, linear diffusion, and key mixing can create or destroy

invariants across rounds. A single round may exhibit obvious symmetries, but repeated composition can either break them or propagate them in disguised form. This makes round-reduced experiments especially useful: by studying truncated versions of a cipher, one can often detect persistent algebraic relations, invariant subspaces, or symmetry-induced biases that are difficult to observe directly in the full construction. Such experiments do not prove insecurity by themselves, but they provide evidence about how closure behaves under iteration.

The chapter also treats group actions as a unifying language for design and cryptanalysis. A group action may encode input/output relabelings, affine transformations, coordinate permutations, or other admissible symmetries of a primitive. From the design perspective, these actions help classify constructions up to natural equivalence. From the cryptanalytic perspective, they can reveal when two apparently different instances are actually the same object in disguise, or when a family of candidates collapses into a smaller number of symmetry classes. The main pitfall is closure: if the set of allowed transformations is not carefully controlled, then an analysis may inadvertently identify non-equivalent objects, or a design may inherit unintended invariants from a larger symmetry group than intended.

Accordingly, the chapter emphasizes a practical distinction between symmetry as a tool and symmetry as a vulnerability. In some contexts, symmetry supports efficient implementation, compact specification, or systematic search over design spaces. In others, it creates fixed points, invariant partitions, or low-dimensional orbits that can be exploited by an attacker. The relevant question is not whether a construction has symmetries, but which symmetries are admissible, which are broken by the round function, and which survive composition in a way that undermines diffusion or confusion.

The accompanying artifact is an equivariance classifier intended to support this analysis. Its role is to categorize candidate transformations and identify whether a given primitive, component, or reduced-round instance is invariant, equivariant, or neither under a specified action. In parallel, round-reduced experiments provide empirical evidence about the persistence of symmetry-related phenomena across a small number of rounds. These artifacts are meant to be reproducible and to support comparison across design candidates rather than to replace formal proofs.

Because cryptographic evaluation can have real-world consequences, this chapter also includes an ethics note. Symmetry analysis and round-reduction experiments should be used responsibly, with attention to disclosure norms, reproducibility, and the limits of empirical evidence. The goal is to improve understanding of cryptographic structure and to support safer design, not to encourage misuse of analytic techniques against deployed systems.

Overall, the chapter situates cryptographic constructions within the book's broader framework of closure, composition, and symmetry. It shows that the same conceptual tools used to study digital hierarchies in general also illuminate the fine structure of S-boxes, round functions, and equivalence relations in modern cryptography.

Section artifact. The section's executable companion is an equivariance classifier for candidate transformations together with a small suite of round-reduced experiments. The intended workflow is to test a primitive against a specified action, record whether the result is invariant, equivariant, or neither, and then compare those outcomes across reduced-round variants to estimate how quickly symmetry-induced structure decays.

Interpretive note. The chapter uses symmetry language in a deliberately conservative way. Equivalence notions such as affine and CCZ equivalence are treated as classification tools, not as security guarantees. Likewise, empirical regularities observed in reduced-round settings are treated

as indicators for further analysis, not as standalone proofs of attack feasibility.

Ethics note. Any implementation of the artifact should be used in accordance with responsible disclosure practices and local policy. The purpose of the analysis is to support robust cryptographic design, reproducible evaluation, and careful interpretation of symmetry effects, especially when empirical findings may be sensitive for deployed systems.

Technical focus. The main technical objects in this chapter are the round function, the S-box equivalence class, and the induced action of admissible symmetries on the space of candidate constructions. The chapter uses these objects to organize three related questions: how quickly $\varepsilon(r)$ decays with round count, which equivalence relations preserve the relevant cryptanalytic metrics, and which round-composition laws preserve or destroy invariants.

Design and analysis workflow. A typical workflow is to begin with a candidate S-box or round function, compute its nonlinearity and differential uniformity, and then compare these values across affine or CCZ-equivalent representatives. One then embeds the component into an SPN or Feistel template, studies the effect of round composition on avalanche and bias, and checks whether any invariant partitions or low-dimensional orbits remain visible under the chosen group action. The resulting evidence can guide both design refinement and cryptanalytic prioritization.

Caution on closure. Symmetry-preserving transformations can be useful for classification, but they can also obscure distinctions that matter for security. In particular, closure under a large transformation family may collapse a design space into a small number of orbits, making it easier to search but also easier for an attacker to exploit shared structure. The chapter therefore treats closure as a property to be measured, not assumed.

Summary. In short, cryptographic constructions live at the intersection of nonlinearity, composition, and symmetry. The chapter's contribution is to make that intersection explicit: to show how S-box symmetries, round composition, and group actions interact, and to provide an artifact-driven method for tracking when those interactions are benign, informative, or dangerous.

3.3.7 Learning on \(\mathbb{B}^n\) and Equivariant Architectures

This chapter studies learning problems on the Boolean hypercube $\mathbb{B}^n$ through the lens of symmetry, spectral structure, and architecture design. The central theme is that many target functions, hypothesis classes, and training objectives become substantially more tractable when one exploits invariances or equivariances under a group action rather than treating inputs as unstructured bit strings.

A useful starting point is the notion of symmetry-aware generalization error. For a hypothesis class constrained by a symmetry group G, one may write the error profile as $\varepsilon(r)$, where r indexes the resolution, radius, or complexity scale at which the learner is evaluated. In this setting, $\varepsilon(r)$ is not merely a scalar performance summary; it records how well the model class captures the orbit structure induced by G, and how rapidly approximation improves as the model is allowed to represent finer symmetry-respecting features. This viewpoint aligns naturally with the book's broader emphasis on closure: the relevant question is not only whether a model fits data, but whether the class of admissible predictors is closed under the transformations that the task itself regards as irrelevant.

Fourier analysis on $\mathbb{B}^n$ provides a canonical language for making these ideas precise. Boolean functions admit expansions in the Walsh–Fourier basis, and symmetry constraints often manifest as sparsity or concentration in particular spectral coordinates. In favorable cases, the target function behaves like a junta, depending effectively on only a small subset of coordinates, or more generally exhibits sparse Fourier support. Such structure can be exploited both for learning and for compression: a learner that identifies the relevant coordinates or dominant coefficients can achieve strong sample efficiency and interpretability. From the perspective of this book, Fourier sparsity is a closure phenomenon in disguise, since the admissible hypothesis class is effectively reduced to a stable subfamily determined by the symmetry and support constraints.

The spectral viewpoint also clarifies why symmetry-aware learning can be robust to overparameterization. When the target lies near a low-degree or low-support Fourier subspace, the effective dimension of the problem is much smaller than 2^n, and the learner can focus on the orbit representatives or coefficient patterns that matter. In practice, this means that the learning pipeline may first identify a symmetry class, then estimate the corresponding spectral mass, and only afterward instantiate a predictor at the desired resolution. The chapter's organizing metric $\varepsilon(r)$ is therefore best read as a multi-resolution error curve: as r increases, the learner is permitted to resolve finer Fourier structure, and the error should decrease in a way that reflects the underlying symmetry class.

Group-equivariant networks provide a constructive architecture for encoding these constraints directly into the model. For permutation symmetries, equivariant layers ensure that permuting the input induces a corresponding permutation of intermediate representations and outputs. More generally, wreath products and related semidirect constructions describe layered symmetry actions that arise when local symmetries are combined with global rearrangements. In such architectures, each layer is designed so that the symmetry constraints are preserved across composition, yielding a network whose entire forward pass respects the chosen group action. This closure across layers is essential: if equivariance is enforced only at the input or output, the internal representation may drift outside the intended symmetry class, weakening both identifiability and generalization.

The closure of constraints across layers also raises identifiability questions. When multiple parameter settings realize the same equivariant map, one must distinguish genuine degrees of freedom from redundant parametrizations induced by symmetry. This is especially important in model families with tied weights, orbit pooling, or structured tensor factorizations. A well-designed equivariant architecture should make the symmetry constraints explicit enough that the learned representation is identifiable up to the unavoidable action of the symmetry group. In practice, this means that the architecture should separate task-relevant variation from symmetry-induced equivalence classes, so that optimization and interpretation remain stable under the group action.

A useful design principle is to treat equivariance as a layerwise invariant rather than a global afterthought. If each layer preserves the relevant group action, then the composition of layers remains inside the same admissible family, and the model can be reasoned about as a closed system. This is particularly important for permutation-equivariant networks and architectures built from wreath products, where the symmetry may act on nested or hierarchical index sets. In those settings, closure across layers is not just aesthetically pleasing; it is what prevents the network from silently encoding spurious asymmetries that would otherwise undermine the intended inductive bias.

The chapter also points toward an artifact-driven workflow. An equivariant model zoo can serve as a catalog of architectures indexed by symmetry type, input domain, and spectral bias. Such a repository is useful not only for benchmarking but also for distillation: a large equivariant model may be compressed into a smaller circuit-like representation that preserves the relevant symmetry behavior while reducing computational cost. This distillation perspective connects learning back

to the book's circuit and closure themes, since the resulting artifact can be analyzed as a finite compositional object with explicit invariants. In a mature workflow, the model zoo is not merely a collection of examples; it is a reproducible design space in which symmetry assumptions, parameter sharing schemes, and compression targets can be compared systematically.

Distillation to circuits is especially valuable when the goal is deployment or auditability. A circuit-style artifact makes the learned symmetry constraints explicit, enabling inspection of which features are preserved, which are discarded, and how the approximation error is distributed across scales. This is consistent with the chapter's emphasis on $\varepsilon(r)$: the distilled artifact should come with a resolution-dependent error profile, not just a single aggregate score. Such a profile helps distinguish coarse but stable approximations from fine-grained models that may be accurate only in a narrow regime.

Privacy considerations enter naturally once symmetry-aware learning is deployed on sensitive data. Symmetry can reduce the effective degrees of freedom of a model, but it can also concentrate information in structured parameters or shared representations. Consequently, one must ask whether equivariant inductive bias improves or weakens privacy guarantees in a given setting. A careful treatment should examine how orbit structure, parameter sharing, and distillation interact with leakage, memorization, and membership inference. The practical lesson is that symmetry is not automatically privacy-preserving; rather, it changes the geometry of the learning problem in ways that must be audited alongside accuracy.

From a systems perspective, privacy analysis should be carried out together with the architecture design. If a model zoo is used to select among symmetry-respecting architectures, then the selection criterion should include not only test accuracy and compression ratio, but also the extent to which the architecture exposes sensitive correlations through shared weights or low-dimensional latent factors. Likewise, if a model is distilled into a circuit-like artifact, the distillation step may either reduce or amplify leakage depending on how faithfully it preserves memorized structure. The right conclusion is not that equivariance is unsafe, but that its privacy impact is architecture-dependent and must be measured explicitly.

Taken together, the chapter develops a unified picture: Fourier structure explains why some Boolean learning problems are compressible; equivariant architectures show how to encode the relevant invariances by design; closure across layers ensures that symmetry constraints survive composition; and artifact-oriented distillation makes the resulting models inspectable and reproducible. The privacy dimension reminds us that every structural gain comes with a corresponding systems-level responsibility. In the language of this book, learning on $\mathbb{B}^n$ is most effective when symmetry, composition, and closure are treated as first-class design principles rather than as after-the-fact regularizers.

3.3.8 Algorithms for Closure & Symmetry Analysis

This chapter collects algorithmic methods for analyzing closure properties and symmetry structure in finite digital objects, with an emphasis on practical decision procedures and reproducible tooling. The central computational tasks are clone membership, generator discovery, canonicalization, equivalence checking, and complexity assessment. In each case, the goal is not only to decide a property, but also to quantify the reliability of the decision procedure through an error profile $\varepsilon(r)$, where the parameter r denotes the chosen resolution, resource budget, or relaxation level. In practice, $\varepsilon(r)$ summarizes soundness and precision, together with observed SAT/SMT success rates under the selected encoding.

A first family of methods addresses clone membership. Given a Boolean function, circuit, or finite operation, one asks whether it belongs to a specified clone or closure class. Two standard

computational encodings are especially useful: SAT-based encodings, which reduce membership to satisfiability of a constraint system, and BDD-based encodings, which exploit canonical decision-diagram structure when the function admits compact representation. In the SAT setting, the membership question is translated into a search for witnesses that violate the defining closure conditions. In the BDD setting, membership can often be checked by structural containment or by testing whether the function can be generated from the prescribed basis under composition. These methods also support generator discovery: rather than merely deciding membership, one searches for a small set of generators whose closure reproduces the target object or class.

Canonicalization is the next major theme. For graph-structured objects and Boolean functions, canonical forms provide a way to compare instances modulo symmetry. Graph isomorphism algorithms supply a canonical labeling or a certificate of equivalence for graph representations of circuits and related combinatorial structures. For Boolean functions, NPN canonicalization is the standard symmetry-reduction problem: one seeks a representative under negation of inputs, permutation of inputs, and negation of the output. Heuristic canonicalization procedures are often necessary in practice, especially when exact search becomes expensive. Typical heuristics include partition refinement, local search over symmetry candidates, signature-based pruning, and incremental refinement from coarse invariants to finer ones. These methods do not replace exact algorithms, but they can dramatically reduce the search space before a final decision step.

Equivalence checking is the unifying decision problem behind these constructions. Two objects are equivalent if they compute the same function, induce the same closure, or lie in the same symmetry orbit under the relevant action. In a SAT-based workflow, equivalence checking is often implemented by constructing a miter or difference formula and asking whether a counterexample exists. In a BDD-based workflow, equivalence can be reduced to pointer equality after canonical reduction, provided the representation is sufficiently compact. For graph-based objects, equivalence may be checked through isomorphism testing or through canonical labels derived from an isomorphism engine. The practical choice among these methods depends on the size of the instance, the expected symmetry, and the cost of the underlying representation.

The complexity-theoretic perspective clarifies why multiple algorithmic strategies are needed. Exact clone membership and equivalence problems can be computationally difficult, and their tractability often depends on parameters such as arity, treewidth, circuit depth, or the size of the symmetry group. Parameterized complexity provides a useful language for this analysis: a problem may be intractable in general while remaining fixed-parameter tractable with respect to a structural measure. This viewpoint is particularly relevant for symmetry analysis, where the presence of large automorphism groups can either help or hinder computation. On the one hand, symmetry can be exploited to compress search; on the other hand, it can create many equivalent branches that must be pruned carefully.

A practical implementation therefore benefits from a layered pipeline. One begins with coarse invariants and cheap filters, then applies SAT, BDD, or graph-isomorphism machinery only when the preliminary tests do not settle the instance. This ordering is especially useful when the benchmark family contains many easy cases mixed with a smaller number of hard instances. In such a pipeline, the reported $\varepsilon(r)$ should be interpreted together with the chosen encoding, the pruning strategy, and the stopping criterion, since these design choices affect both the observed success rate and the reliability of the final answer.

The chapter therefore treats algorithms and complexity as two sides of the same design problem. A good implementation should expose the tradeoff between exactness, scalability, and certification. The metric $\varepsilon(r)$ is used here as a compact summary of that tradeoff: lower values indicate better soundness and precision at the chosen resource level, while higher SAT/SMT success rates indicate that the encoding is effective on the target benchmark family. In a reproducible workflow, these

quantities are reported together with instance size, symmetry profile, and solver configuration.

The accompanying artifact is a CLI toolkit with benchmarks. The toolkit is intended to support batch evaluation of clone membership, canonicalization, and equivalence-checking pipelines on a curated set of examples. A typical command-line workflow includes loading an instance, selecting an encoding strategy, running the solver or canonicalizer, and exporting a machine-readable report containing the decision result, runtime, resource usage, and the observed $\varepsilon(r)$ summary. Benchmark output should be stable enough to support regression testing and cross-method comparison, so that improvements in heuristics or encodings can be measured against a fixed baseline.

Taken together, these methods provide a practical bridge between the abstract closure and symmetry theory developed earlier in the book and the executable analyses needed in later case studies. They show how clone-theoretic questions, symmetry reduction, and equivalence checking can be turned into concrete algorithms, while preserving a disciplined account of correctness and computational cost.

3.3.9 Case Studies & Build Logs

This chapter records build logs at multiple resolutions, from function-level specifications to gate-level realizations, timing behavior, and device-level constraints. The organizing idea is the error profile $\varepsilon(r)$, viewed as a resolution-dependent discrepancy measure that can be tracked across successive refinement steps. In practice, the same design may be inspected as a Boolean function, a gate network, a timing-constrained netlist, or a physical device artifact; the build log should preserve the correspondence among these views so that regressions, optimizations, and closure failures can be localized precisely.

Multi-resolution dashboards. A useful build dashboard exposes at least four linked layers: function, gate, timing, and device. At the function layer, one records the intended truth table, algebraic normal form, or specification fragment. At the gate layer, one records the chosen decomposition, including shared subexpressions and structural symmetries. At the timing layer, one records critical paths, slack, and any retiming or buffering decisions. At the device layer, one records the implementation substrate and the constraints that arise from it, such as fan-out limits, placement effects, or technology-specific primitives. The purpose of the dashboard is not merely reporting; it is to make refinement steps auditable and to expose where a design ceases to be stable under further composition. In this sense, $\varepsilon(r)$ is not just a score but a diagnostic trace: as the resolution changes, the build log should show which discrepancies disappear, which persist, and which are introduced by the act of refinement itself.

From NAND to a 4-bit ALU to a microcontroller. The chapter uses a progression of increasingly rich artifacts as a recurring case-study pattern. The first stage is a NAND-centered construction, where a small basis is expanded into derived gates and then into larger combinational blocks. The second stage is a 4-bit ALU, where arithmetic and logic operations are assembled from reusable slices and control logic. The third stage is a microcontroller-scale integration, where datapath, control, memory interface, and timing closure interact. Across these stages, the same questions recur: which symmetries survive the refinement, which optimizations preserve semantics, and which implementation choices introduce new constraints that must be tracked in the build log. The point of the progression is not only to scale up complexity, but to show that the same bookkeeping discipline applies from a single gate family to a system-on-chip-style artifact.

Compression by symmetry. A central optimization theme is compression by symmetry, especially through LUT packing and factoring. When a family of subfunctions is related by permutation, complementation, or other structural symmetries, the implementation can often share resources rather than duplicating them. LUT packing exploits the fact that multiple small functions may fit into a single configurable resource when their inputs and outputs are arranged compatibly. Factoring reduces repeated structure by extracting common subexpressions or common control patterns. In both cases, the build log should record the symmetry used, the transformation applied, and the resulting effect on area, delay, and maintainability. The key point is that optimization is not only a local improvement; it is a claim that the compressed artifact remains equivalent to the original specification under the chosen notion of refinement. When the compression is successful, the log should make clear whether the gain came from a true symmetry of the specification or from an incidental regularity of the chosen encoding.

Post-mortems and closure failures. Not every refinement succeeds. This chapter therefore includes post-mortems for closure failures, meaning cases where an intended compositional law or symmetry-preserving transformation does not hold in the implemented artifact. Such failures are valuable because they often produce counterexamples that clarify the boundary of a theorem, a synthesis rule, or a design heuristic. A post-mortem should identify the failed assumption, the smallest counterexample, the resolution at which the failure first appears, and the corrective action, if any. In the context of this book, counterexamples are not merely negative results; they are diagnostic artifacts that refine the closure story by showing exactly where the hierarchy stops being stable. A good post-mortem also distinguishes logical failure from physical failure: some counterexamples arise because the specification was too strong, while others appear only after timing, placement, or device constraints are taken into account.

Artifact index and reproducibility. The chapter also serves as an artifact index. It should point to the repository structure, schema definitions, and toolchain specifications needed to reproduce the builds described in the logs. The repository index identifies where specifications, generated netlists, timing reports, and validation scripts live. The schemas define the expected shape of intermediate artifacts so that logs can be parsed and compared mechanically. The toolchain specification records versions, synthesis settings, and any environment assumptions required for reproducibility. Together, these materials make the chapter a bridge between narrative explanation and executable evidence. A reader should be able to move from the prose description of a build to the exact files and commands that generated it, and then back again through the logged resolutions r and discrepancy values $\varepsilon(r)$.

Open problems. Several open problems remain intentionally visible. How should $\varepsilon(r)$ be normalized across heterogeneous resolutions so that function-level and device-level discrepancies can be compared meaningfully? Which symmetry classes admit the most effective LUT packing without sacrificing timing closure? When does factoring improve area but worsen critical path behavior, and how should such trade-offs be represented in a build log? What is the right abstraction for closure failures that arise only after physical implementation, rather than at the logical or gate level? These questions are left open as prompts for further experimentation and for future artifact-driven refinement. They also mark the boundary between what can be asserted abstractly and what must be established empirically in a concrete toolchain.

Chapter use. Readers may treat this chapter as a logbook template: each case study should state the specification, the refinement path, the optimization or failure observed, and the reproducibility artifacts required to replay the result. In that sense, the chapter is less a single theorem than a disciplined record of how the book's abstract framework behaves when pushed through concrete builds. The intended workflow is iterative: record the artifact, measure $\varepsilon(r)$, attempt compression or refinement, and then preserve both successes and failures as part of the permanent build history.

Repository and schema placeholders. For reproducibility, the chapter should be paired with a repository index and machine-readable schemas. A minimal index may include entries for specification sources, generated netlists, timing reports, device constraints, and validation scripts. Likewise, the schema layer should describe the expected fields for each artifact class so that build logs can be checked automatically. In the present draft, these references are intentionally left as placeholders to be filled by the project-specific repository layout and toolchain manifest.

Build-log template. A typical entry in the log may record: the source specification, the target resolution r, the transformation applied, the measured discrepancy $\varepsilon(r)$, and any observed trade-off in area, delay, or closure. When a transformation fails, the same template should capture the counterexample, the first failing resolution, and the reason the failure matters for later refinement steps. This template is meant to be reused across the NAND, ALU, and microcontroller case studies so that the chapter reads as a coherent sequence of comparable experiments rather than as isolated anecdotes.

4 Bibliography

References

[1] J. Adámek, H. Herrlich, and G. E. Strecker, *Abstract and Concrete Categories: The Joy of Cats*. Wiley, 1990.

[2] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed. Springer, 1998.

[3] S. Eilenberg, *Automata, Languages, and Machines*, Vol. A. Academic Press, 1974.

[4] E. L. Post, "The Two-Valued Iterative Systems of Mathematical Logic," *Annals of Mathematics Studies*, no. 5, Princeton University Press, 1941.

[5] E. Börger, E. Grädel, and Y. Gurevich, *The Classical Decision Problem*. Springer, 2001.

[6] P. Jeavons, D. Cohen, and M. Gyssens, "Closure properties of constraints," *Journal of the ACM*, vol. 44, no. 4, pp. 527–548, 1997.

[7] S. Burris and H. P. Sankappanavar, *A Course in Universal Algebra*. Springer, 1981.

[8] F. Bergeron, G. Labelle, and P. Leroux, *Combinatorial Species and Tree-like Structures*. Cambridge University Press, 1998.

[9] A. Joyal, "Une théorie combinatoire des séries formelles," *Advances in Mathematics*, vol. 42, no. 1, pp. 1–82, 1981.

[10] R. Street, "Quantum groups: a path to current algebra," in *The Proceedings of the 1986 AMS Summer Research Institute*, 2007 reprint and related categorical foundations.

[11] S. Lack, "A 2-categories companion," in *Towards Higher Categories*, Springer, 2010.

[12] P. Selinger, "A survey of graphical languages for monoidal categories," in *New Structures for Physics*, Springer, 2011, pp. 289–355.

[13] B. Coecke and A. Kissinger, *Picturing Quantum Processes*. Cambridge University Press, 2017.

[14] P. Selinger, "Towards a quantum programming language," *Mathematical Structures in Computer Science*, vol. 14, no. 4, pp. 527–586, 2004.

[15] T. Hales, "Formal proof," *Notices of the AMS*, vol. 55, no. 11, pp. 1370–1380, 2008.

[16] G. Gonthier, "Formal proof—the four-color theorem," *Notices of the AMS*, vol. 55, no. 11, pp. 1382–1393, 2008.

[17] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.

[18] M. Huth and M. Ryan, *Logic in Computer Science: Modelling and Reasoning about Systems*, 2nd ed. Cambridge University Press, 2004.

[19] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.

[20] J.-Y. Girard, Y. Lafont, and P. Taylor, *Proofs and Types*. Cambridge University Press, 1989.

[21] P. T. Johnstone, *Stone Spaces*. Cambridge University Press, 1982.

[22] G. Birkhoff, "Lattice theory," *American Mathematical Society Colloquium Publications*, 1940.

[23] B. D. McKay, "Practical graph isomorphism," *Congressus Numerantium*, vol. 30, pp. 45–87, 1981.

[24] L. Babai, "Graph isomorphism in quasipolynomial time," in *Proceedings of the 48th ACM Symposium on Theory of Computing*, 2016, pp. 684–697.

[25] R. Diestel, *Graph Theory*, 5th ed. Springer, 2017.

[26] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[27] C. E. Shannon and W. Weaver, *The Mathematical Theory of Communication*. University of Illinois Press, 1949.

[28] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.

[29] O. Goldreich, *Foundations of Cryptography*, Vol. 1. Cambridge University Press, 2001.

[30] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed. Chapman & Hall/CRC, 2005.

[31] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[32] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, "Geometric deep learning: Grids, groups, graphs, geodesics, and gauges," *arXiv:2104.13478*, 2021.

[33] R. Wille and R. Drechsler, *Towards a Design Flow for Reversible Logic*. Springer, 2009.

[34] M. Amy, D. Maslov, M. Mosca, and M. Roetteler, "A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 6, pp. 818–830, 2013.

[35] The Agda Team, *Agda Documentation*. Available as project documentation; accessed for formalization patterns.

[36] The Coq Development Team, *The Coq Proof Assistant Reference Manual*. Available as project documentation; accessed for proof engineering patterns.

5 Back Matter

5.1 Appendices

This appendix collects compact reference material used throughout the book. It is organized as a set of self-contained guides and tables intended for quick consultation rather than extended exposition.

- **Appendix A: Post's lattice cheat sheet and completeness tables.** A reference summary of the principal clone-theoretic classes, their closure properties, and the standard completeness criteria used in the Boolean setting.
- **Appendix B: Operad and PROP identities, with coherence diagrams.** A compact list of the identities, interchange laws, and diagrammatic coherence principles that recur in the operadic and PROP-based constructions.
- **Appendix C: ZX, traced monoidal, and decorated cospan guides.** A practical guide to the notation, rewrite rules, and categorical conventions used for ZX-calculus, traced monoidal structure, and decorated cospan formalisms.
- **Appendix D: Metrics catalog for $\varepsilon(r)$.** A catalog of metric definitions, units, and calibration conventions for the error and resolution measures denoted by $\varepsilon(r)$, including the bookkeeping needed to compare values across chapters and build artifacts.

The appendices are meant to support cross-referencing and reproducibility. When a later chapter invokes a lattice class, an operadic law, a traced feedback identity, or a metric convention, the corresponding reference material should be located here before consulting the main text.

For consistency with the rest of the book, notation in these appendices follows the conventions established in the front matter. Where a result is stated in abbreviated form, the surrounding chapters provide the full derivation or proof sketch; the appendix entry serves as the canonical lookup location for the statement itself.

Usage note. These materials are intentionally terse. They are designed to be stable reference points for proofs-as-builds, implementation checks, and calibration workflows, and they may be expanded in future revisions as the surrounding chapters evolve.

Appendix A: Post's lattice cheat sheet and completeness tables. This appendix records the standard Boolean clone classes, their defining closure properties, and the usual completeness criteria used to recognize functionally complete sets. It is intended as a lookup table for the lattice-theoretic terminology used in the chapters on closure and clones.

Appendix B: Operad and PROP identities. This appendix summarizes the basic operadic identities, the PROP interchange law, and the coherence patterns that justify diagrammatic manipulations. It is the reference point for equations and string-diagram equalities that are used without rederivation in the main text.

Appendix C: ZX, traced monoidal, and decorated cospan guides. This appendix collects the notation and rewrite conventions for ZX-calculus, the axioms and trace identities for traced monoidal categories, and the bookkeeping conventions for decorated cospans. It is meant to make the categorical and diagrammatic chapters easier to read and verify.

Appendix D: Metrics catalog for $\varepsilon(r)$. This appendix lists the metric definitions, units, normalization choices, and calibration procedures associated with $\varepsilon(r)$. It provides the comparison conventions needed to interpret error values consistently across chapters, experiments, and build artifacts.

- The appendices are reference-oriented rather than tutorial-oriented.
- Each entry is intended to be stable under later chapter revisions.
- Cross-references should point here whenever a definition, identity, or calibration rule is used repeatedly.

Appendix A: Post's lattice cheat sheet; completeness tables. The Boolean reference tables in this appendix summarize the standard clone classes appearing in Post's lattice, together with the closure properties that characterize them. The tables are intended to support quick checks of whether a given set of Boolean operations is closed under composition, preserves a relation, or satisfies one of the standard completeness criteria.

Appendix B: Operad/PROP identities; coherence diagrams. This appendix records the identities and coherence principles that are used repeatedly in the operad and PROP chapters. In particular, it serves as a compact reference for interchange laws, associativity and unit constraints, and the diagrammatic equalities that justify rewriting composite operations.

Appendix C: ZX, traced monoidal, decorated cospan guides. This appendix gathers the working conventions for ZX-calculus, traced monoidal categories, and decorated cospans. It includes the notation and rewrite rules needed to interpret string diagrams, the feedback identities used in traced settings, and the compositional bookkeeping used for decorated cospan constructions.

Appendix D: Metrics catalog for $\varepsilon(r)$: defs, units, calibration. This appendix catalogs the metric definitions associated with $\varepsilon(r)$, together with their units, normalization choices, and calibration procedures. The intent is to make comparisons across chapters and build artifacts unambiguous by fixing the conventions used to report and interpret error values.

- Definitions are stated in a lookup-friendly form.
- Units and normalization choices are recorded alongside each metric.
- Calibration conventions are included so that reported values can be compared consistently.

5.2 Glossary & Symbol Index

This back-matter section serves two purposes: it provides plain-language definitions for recurring terms, and it gives a bidirectional map from symbols to the chapters and sections where they are introduced, used, or proved. The intent is to make the book navigable for readers who want to move from notation to meaning, and from a symbol back to the relevant proof, construction, or build artifact.

Glossary. Concise definitions of the book's recurring technical terms are given in reader-facing language. Each entry should point, where appropriate, to the chapter, section, proposition, theorem, or build artifact in which the term is first defined or most substantially developed.

Symbol index. A symbol-to-location map records where each notation item appears in the book. The map is bidirectional in the sense that readers can locate a symbol from its meaning, and can also recover the symbol from the chapter or section in which it is used.

Cross-references. Definitions and symbol entries should cross-reference proofs, constructions, and worked examples rather than restating them. This section is therefore a navigational layer, not a substitute for the main exposition.

Glossary entries. The following terms are representative of the vocabulary used throughout the book; additional entries may be added as the manuscript stabilizes.

Closure. A property of a class of objects or operations indicating that the class is stable under a specified operation. In this book, closure is used in the sense of algebraic, categorical, and computational stability conditions.

Composition. The operation of combining processes, maps, circuits, or morphisms so that the output of one becomes the input of another. Composition is the organizing principle of the book's hierarchy of constructions.

Symmetry. A transformation that preserves the relevant structure of an object, function, circuit, or system. Symmetry may appear as an automorphism, an invariance, an equivariance condition, or a group action.

Clone. A set of Boolean functions closed under composition and containing projections. Clones are central to the discussion of Post's lattice and functional completeness.

Operad. An algebraic structure encoding multi-input composition with typed arities. Operads provide a language for compositional hierarchies and modular assembly.

PROP. A symmetric monoidal category-like formalism for operations with multiple inputs and outputs. PROPs are used to model circuit-like and process-like composition.

Trace. A feedback operator that closes a wire or channel to form a loop. Traces are used to formalize sequential composition with feedback.

Equivariance. Compatibility of a map or model with a symmetry action. Equivariant architectures preserve symmetry information by construction.

Reproducibility. The property that a claim can be rebuilt, checked, or rerun from the stated artifacts, scripts, and inputs. In this book, reproducibility is treated as part of the mathematical workflow.

Symbol index. The table below is a placeholder structure for the bidirectional symbol map. It should be populated with the final manuscript's notation and the exact chapter, section, and page locations once pagination is fixed.

Symbol	Meaning	Location(s)
$\mathbb{B}$	Boolean domain	Chapter 3; Chapter 7; Chapter 14
$\mathbb{B}^n$	n-fold Boolean cube	Chapter 1; Chapter 14; Chapter 15
$\circ$	Composition of maps or morphisms	Chapters 1–16
$\otimes$	Tensor or parallel composition	Chapters 2, 5, 12
$\mathrm{Cl}(\cdot)$	Closure operator or closure of a set	Chapter 3; Chapter 15
$\mathrm{Sym}(\cdot)$	Symmetry group or symmetry set	Chapter 8; Chapter 13
Tr	Trace / feedback operator	Chapter 5; Appendix material on traced monoidal stru
NPN	Negation-permutation-negation equivalence	Chapter 8; Chapter 15

Usage note. For each symbol, the index should list the earliest defining location, the principal development location, and any later sections where the notation is reused in a materially different context. When page numbers are available, they should be included alongside chapter and section references to support fast lookup in the printed edition.

Editorial convention. If a term has both an informal and a formal meaning in different chapters, the glossary entry should state the informal meaning first and then point to the formal definition. If a symbol is overloaded, the index should disambiguate it by context rather than by inventing new notation unless the manuscript explicitly introduces one.

Revision note. To align this section more tightly with the rest of the manuscript, the glossary should remain concise and reader-facing, while the symbol index should be expanded only after the notation in Chapters 1–16 is finalized. In particular, the final pass should add section-level and page-level references for the most frequently reused symbols, and should distinguish between definitions, principal uses, and proof locations where those differ.

5.3 Artifacts & Reproducibility

Manifest A reproducibility manifest records the code and data artifacts together with their content hashes, so that each referenced object can be identified unambiguously. The manifest should also enumerate the build scripts used to generate derived outputs and the container images or other execution environments required to rerun the workflow. In the manuscript's claims-as-builds workflow, the manifest serves as the authoritative index from published claims to the exact artifact revisions that support them.

Harnesses Reproducibility harnesses package the test descriptions, continuous integration configuration, and recorded outputs that demonstrate how the book's executable artifacts are validated. In practice, this includes the commands or test cases used to exercise the build, the CI jobs that automate those checks, and the resulting logs or reports produced by the pipeline. The harnesses should be written so that a reader can rerun the same checks locally or in CI and compare the new outputs against the archived evidence.

For this book, the reproducibility record is intended to make the claims-as-builds workflow inspectable end to end: a reader should be able to locate the manifest, reconstruct the environment, rerun the harnesses, and compare the resulting outputs against the archived CI evidence. Where

multiple artifact versions exist, the manifest should indicate the exact revision associated with the published build. Together, the manifest and harnesses provide the minimal audit trail needed to distinguish source material, generated outputs, and the execution context that links them.

In practical terms, the reproducibility package should include the following items:

- a manifest of source code, data, and derived artifacts with content hashes;
- the build scripts or commands used to regenerate the published outputs;
- container definitions or equivalent environment specifications;
- test descriptions and harnesses that exercise the build;
- continuous integration configuration for automated verification; and
- archived outputs, logs, or reports that document the observed results.

This section is intentionally operational rather than theoretical: it defines the evidence needed to rerun, inspect, and verify the book's executable claims without ambiguity.

5.4 Errata & Change Log

This section records corrections, version history, limitations, and open problems for the present draft.

Corrections. Corrections are tracked as they are identified during drafting, review, and build verification. Each substantive change should record the affected location, the corrected statement, and the reason for the revision so that the manuscript remains auditable. When a correction changes a claim, definition, notation, dependency, or cross-reference elsewhere in the book, the corresponding downstream sections should be updated to preserve consistency. Minor typographical fixes may be noted briefly, while substantive mathematical or structural corrections should be described explicitly.

Version history. The change log serves as a compact history of the section and, where relevant, the book-level draft. Each entry should include a version identifier, a date, and a concise summary of the changes introduced in that revision. Entries should be ordered chronologically so that the evolution of the manuscript can be reconstructed from the log alone.

Limitations. The present draft is intentionally limited to the information currently available in the source material. It does not yet enumerate specific errata items, nor does it provide a complete release-by-release history. Any claim of completeness should therefore be read as provisional until the section is expanded with concrete entries. In particular, this text describes the intended maintenance policy rather than a finalized archive of corrections.

Open problems. Open problems include unresolved technical issues, pending editorial questions, and any known gaps in coverage that remain after review. Each item should be listed explicitly when identified, together with a short note on whether the issue is conceptual, notational, computational, or bibliographic in nature. Where possible, each open problem should be paired with a proposed next step or a pointer to the section or chapter most directly affected.

For maintenance purposes, future entries in this section should distinguish between minor typographical fixes, substantive mathematical corrections, and unresolved limitations that may affect interpretation or implementation. When a limitation is resolved, it should be moved from the open-problems list into the correction history with a brief note on the resolution path.

The imported source material for this section is minimal and confirms only the intended categories: corrections, version history, limitations, and open problems. Accordingly, this draft preserves the maintenance policy in prose form and leaves concrete errata entries to future revisions.

Tracked corrections. No specific corrections have been recorded in the imported source material yet. When entries are added, each should identify the affected location, the corrected statement, and the downstream impact, if any.

Tracked version history. No dated revision table is available in the current source material. A future log should include version identifiers, dates, and concise summaries of changes in chronological order.

Known limitations. The present section is a placeholder for a fuller errata archive. It does not yet enumerate known issues, and it should not be read as a claim that the manuscript is free of defects.

Open problems. Any unresolved issues should be listed here with enough detail to support follow-up. At minimum, each item should indicate whether it is conceptual, notational, computational, or bibliographic, and should point to the section or chapter most directly affected.